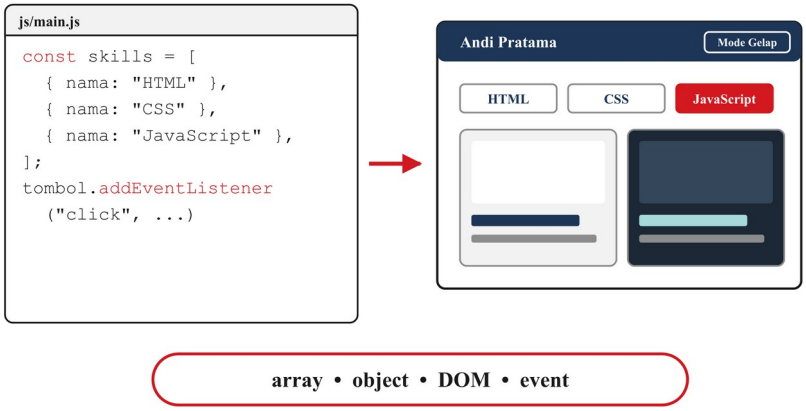
MODUL KELAS INDUSTRI

ARRAY, OBJECT, DOM, DAN EVENT

Paket 1 Full Stack Basic, Minggu 07 (P1-W07)



Kelas X Semester 1

Kompetensi Keahlian Pengembangan Perangkat Lunak dan Gim

Disusun oleh:

Tim Kelas Industri UBIG

PROGRAM KELAS INDUSTRI

PT UNIVERSAL BIG DATA

Tahun Ajaran 2026/2027

KATA PENGANTAR

Selamat datang di modul ketujuh Paket 1 *Full Stack Basic* Kelas Industri PT Universal Big Data (UBIG). Minggu lalu kamu menulis program pertamamu: menyimpan nilai dalam variabel, menghitung, memilih dengan `if`, dan membungkusnya dalam fungsi. Namun, semua hasilnya baru tampil di Console. Pengunjung portofoliomu belum merasakan apa pun.

Minggu ini JavaScript mulai bekerja di halaman. Kamu akan menyimpan banyak data sekaligus dalam **array** dan **object**, mengubah isi halaman melalui **DOM**, dan menanggapi klik serta ketikan pengunjung dengan **event**. Hasilnya, daftar keahlian di Beranda tersusun otomatis dari data, portofolio punya tombol mode gelap, form Kontak menghitung karakter pesan, dan halaman Proyek bisa disaring per kategori.

Kodenya mungkin terlihat lebih panjang dari minggu lalu, tetapi setiap barisnya dibangun dari hal yang sudah kamu kenal: variabel, fungsi, *arrow function*, `if`, dan *template literal*. Kerjakan tahap demi tahap, periksa hasilnya di halaman dan di Console, lalu lanjutkan ke tahap berikutnya.

Malang, Juli 2026
Tim Kelas Industri UBIG

DAFTAR ISI

KATA PENGANTAR.....i
DAFTAR ISI.....ii
DAFTAR GAMBAR.....vi
DAFTAR TABEL.....viii
PETUNJUK PENGGUNAAN MODUL.....ix
Petunjuk bagi Siswa.....ix
Petunjuk bagi Guru dan Mentor.....ix
BAB I PENDAHULUAN.....1
1.1 Deskripsi Modul.....1
1.2 Prasyarat.....1
1.3 Tujuan Pembelajaran.....2
1.4 Peta Konsep.....2
1.5 Alokasi Waktu.....3
BAB II KEGIATAN PEMBELAJARAN 1: ARRAY.....4
2.1 Tujuan Kegiatan Pembelajaran.....4
2.2 Uraian Materi.....4
2.2.1 Satu Variabel untuk Banyak Data.....4
2.2.2 Membuat Array dan Mengambil Isinya.....5
2.2.3 length dan push.....6
2.2.4 Perulangan for...of.....7
2.3 Praktik: Memulai main.js.....8
2.3.1 Tahap 1: Melepas latihan.js dan logika.js.....8
2.3.2 Tahap 2: Membuat main.js di Ketiga Halaman.....8
2.3.3 Tahap 3: Array Keahlian.....9
2.4 Rangkuman.....10
2.5 Latihan.....10
BAB III KEGIATAN PEMBELAJARAN 2: OBJECT DAN ARRAY OF OBJECTS. 11
3.1 Tujuan Kegiatan Pembelajaran.....11
3.2 Uraian Materi.....11
3.2.1 Object: Satu Data dengan Banyak Keterangan.....11

3.2.2 Membaca dan Mengubah Properti....12
3.2.3 Array of Objects....12
3.2.4 forEach, map, dan filter....13
3.3 Praktik: Keahlian sebagai Array of Objects....15
3.3.1 Tahap 1: Menambahkan Level Keahlian....15
3.3.2 Tahap 2: Memeriksa Data di Console....15
3.4 Rangkuman....16
3.5 Latihan....17
BAB IV KEGIATAN PEMBELAJARAN 3: DOM....18
4.1 Tujuan Kegiatan Pembelajaran....18
4.2 Uraian Materi....18
4.2.1 Pohon DOM....18
4.2.2 Mengambil Elemen dengan querySelector....19
4.2.3 Mengubah Isi: textContent dan innerHTML....19
4.2.4 Membuat Elemen Baru: createElement dan appendChild....20
4.2.5 Mengatur Class dengan classList....22
4.3 Praktik: Daftar Keahlian Dinamis....22
4.3.1 Tahap 1: Mengosongkan Daftar di HTML....22
4.3.2 Tahap 2: Menampilkan Keahlian dengan createElement....23
4.3.3 Tahap 3: Membuktikan Daftar Berasal dari Array....24
4.3.4 Tahap 4: Memperbaiki Error di Halaman Lain....24
4.4 Rangkuman....25
4.5 Latihan....26
BAB V KEGIATAN PEMBELAJARAN 4: EVENT DAN MODE GELAP....27
5.1 Tujuan Kegiatan Pembelajaran....27
5.2 Uraian Materi....27
5.2.1 Event dan addEventListener....27
5.2.2 Event click....28
5.2.3 Event input....28
5.2.4 Variabel CSS untuk Tema....29
5.3 Praktik A: Tombol Mode Gelap....30
5.3.1 Tahap 1: Menambahkan Tombol di Header....30

5.3.2 Tahap 2: Mencoba classList.toggle....31
5.3.3 Tahap 3: Mengubah Warna Menjadi Variabel CSS....32
5.3.4 Tahap 4: Mengganti Teks Tombol....35
5.4 Praktik B: Penghitung Karakter Pesan....36
5.4.1 Tahap 1: Batas dan Tempat Penghitung....36
5.4.2 Tahap 2: Menghitung dengan Event input....37
5.5 Rangkuman....38
5.6 Latihan....38
BAB VI KEGIATAN PEMBELAJARAN 5: DAFTAR PROYEK DINAMIS DAN FILTER....39
6.1 Tujuan Kegiatan Pembelajaran....39
6.2 Uraian Materi....39
6.2.1 Data Proyek dalam Satu Array....39
6.2.2 Membuat Kartu dari Data....40
6.2.3 map dan join: Banyak Data Menjadi Banyak Kartu....40
6.2.4 filter untuk Memilih Kategori....41
6.3 Praktik: Halaman Proyek Dinamis....41
6.3.1 Tahap 1: Menyiapkan Gambar Proyek Baru....42
6.3.2 Tahap 2: Wadah Kartu dan Tombol Filter....42
6.3.3 Tahap 3: Data Proyek....43
6.3.4 Tahap 4: Menampilkan Kartu....45
6.3.5 Tahap 5: Menjalankan Filter....46
6.3.6 Tahap 6: Kode Lengkap main.js....48
6.3.7 Tahap 7: Commit dan Push....49
6.4 Memeriksa Hasil Kerja....52
6.5 Rangkuman....52
6.6 Latihan....52
6.7 Tugas dan Pengumpulan Minggu Ini....53
6.7.1 Tugas Rumah....53
6.7.2 Tantangan (Opsional)....54
6.7.3 Kriteria Selesai....55
6.7.4 Ketentuan Pengumpulan P1-W07....55

6.7.5 Urutan Mengerjakan Tugas.....57
BAB VII EVALUASI.....59
7.1 Soal Pilihan Ganda.....59
7.2 Soal Uraian.....61
7.3 Soal Praktik.....61
7.4 Rubrik Penilaian Tugas.....62
BAB VIII PENUTUP.....63
GLOSARIUM.....64
DAFTAR PUSTAKA.....65

DAFTAR GAMBAR

Gambar 1.1 Peta konsep Modul P1-W07.....2
Gambar 2.1 Array seperti rak berlaci bernomor.....5
Gambar 2.2 Membuat array dan mengambil isinya dengan indeks.....5
Gambar 2.3 Menambah isi array dengan push.....6
Gambar 2.4 Cara kerja perulangan for...of.....7
Gambar 2.5 Hasil perulangan for...of di Console.....8
Gambar 2.6 File main.js di VS Code.....9
Gambar 2.7 Hasil array keahlian di Console.....10
Gambar 3.1 Object seperti kartu pelajar.....11
Gambar 3.2 Membaca dan mengubah properti object.....12
Gambar 3.3 Array of objects seperti tabel.....13
Gambar 3.4 Perbedaan forEach, map, dan filter.....13
Gambar 3.5 Hasil forEach, map, dan filter di Console.....14
Gambar 3.6 Array keahlian sebagai tabel di Console.....16
Gambar 4.1 Pohon DOM halaman Beranda.....18
Gambar 4.2 Mengambil elemen dengan querySelector.....19
Gambar 4.3 Mengubah judul dan paragraf dari Console.....20
Gambar 4.4 Tiga langkah menambahkan elemen baru.....21
Gambar 4.5 Paragraf baru ditambahkan dari Console.....21
Gambar 4.6 Daftar keahlian dibuat dari array.....23
Gambar 4.7 Elemen li hasil JavaScript di panel Elements.....24
Gambar 4.8 TypeError karena elemen tidak ada di halaman Proyek.....24
Gambar 5.1 Bagian-bagian addEventListener.....27
Gambar 5.2 Hasil event click setelah tombol diklik tiga kali.....28
Gambar 5.3 Hasil event input saat nama diketik.....29
Gambar 5.4 Variabel CSS untuk tema terang dan gelap.....30
Gambar 5.5 Tombol Mode Gelap di header.....31
Gambar 5.6 Percobaan pertama: hanya latar yang berubah.....32
Gambar 5.7 Mode gelap berlaku di semua bagian halaman.....34
Gambar 5.8 Teks tombol berubah menjadi Mode Terang.....35

Gambar 5.9 Mode gelap di halaman Kontak pada layar ponsel.....36
Gambar 5.10 Penghitung karakter pesan di halaman Kontak.....38
Gambar 6.1 Alur dari data proyek ke kartu di halaman.....41
Gambar 6.2 Mencoba data proyek dari Console.....41
Gambar 6.3 Halaman Proyek berisi enam kartu dari array.....47
Gambar 6.4 Filter JavaScript menampilkan dua proyek.....47
Gambar 6.5 Halaman Proyek dinamis di layar ponsel.....48
Gambar 6.6 git status setelah praktik Minggu 07.....50
Gambar 6.7 Commit dan push praktik Minggu 07.....51
Gambar 6.8 Filter proyek di website live.....51
Gambar 6.9 Contoh hasil tugas: filter CSS dengan tombol aktif.....54
Gambar 6.10 Contoh tangkapan layar pertama: mode terang.....56
Gambar 6.11 Contoh tangkapan layar kedua: mode gelap.....57
Gambar 6.12 Contoh tangkapan layar ketiga: filter di website live.....57

DAFTAR TABEL

Tabel 1.1 Alokasi waktu Minggu 07....3
Tabel 3.1 Tiga method array....14
Tabel 4.1 Method classList....22
Tabel 5.1 Warna tetap yang diganti variabel di style.css....33
Tabel 6.1 Enam proyek di halaman Proyek....40
Tabel 6.2 Tiga gambar proyek baru....42
Tabel 6.3 Cara memeriksa hasil kerja Minggu 07....52
Tabel 6.4 Hasil filter yang benar....53
Tabel 6.5 Petunjuk menandai tombol aktif....54
Tabel 6.6 Petunjuk tantangan localStorage....55
Tabel 6.7 Ketentuan pengumpulan P1-W07....56
Tabel 7.1 Rubrik penilaian tugas P1-W07....62

PETUNJUK PENGGUNAAN MODUL

Petunjuk bagi Siswa

1. Siapkan folder `portofolio` hasil tugas Minggu 06 (berisi `js/latihan.js` dan `js/logika.js`) yang sudah tersimpan di GitHub dan tampil di Vercel. Seluruh praktik minggu ini melanjutkan folder tersebut.
2. Pelajari Kegiatan Pembelajaran secara berurutan dari 1 sampai 5. Semua kode halaman ditulis di satu file baru, `js/main.js`, yang dipasang di ketiga halaman.
3. Ketik sendiri setiap contoh kode, simpan dengan **Ctrl+S**, lalu periksa hasilnya di halaman dan di panel Console (**Ctrl+Shift+J**). Bandingkan dengan gambar di modul. Jika berbeda, baca pesan di Console dan kotak **Kesalahan Umum**.
4. Kotak **Coba Sendiri** berisi percobaan singkat di Console. Lakukan semuanya, karena percobaan kecil inilah yang membangun pemahaman.
5. Jawab Latihan di akhir setiap kegiatan, kerjakan Tugas dan Pengumpulan Minggu Ini, lalu pelajari bab Evaluasi: soal pilihan ganda dan uraiannya menjadi kuis di awal pertemuan P1-W08, sedangkan soal praktiknya dikerjakan sebagai remedial jika tugasmu belum memenuhi kriteria selesai. Format laporan, standar video, tenggat, sanksi, serta ketentuan kuis dan remedial tercantum di dokumen **Panduan dan Ketentuan Kelas Industri UBIG**.

Petunjuk bagi Guru dan Mentor

1. Gunakan modul ini bersama presentasi P1-W07 (urutannya sama dengan modul) dan dokumen Kunci Jawaban P1-W07 yang khusus untuk mentor.
2. Gunakan 15 menit pembuka untuk kuis P1-W06 (10 menit, Google Form), lalu ulas singkat fungsi `hitungRataRata` dan `predikat` dari minggu lalu karena keduanya dipakai sebagai contoh data proyek.
3. Tekankan cara memeriksa hasil JavaScript di halaman: muat ulang, lihat Console, dan periksa elemen baru di panel Elements. Pastikan setiap siswa memiliki daftar keahlian dari array dan tombol mode gelap sebelum pertemuan berakhir.
4. Siapkan Google Form kuis dari soal pilihan ganda dan uraian bab Evaluasi untuk pembuka pertemuan P1-W08. Kunci, skor, dan daftar periksa remedial ada di Kunci Jawaban P1-W07.

5. Tandai butir kompetensi *Fondasi: DOM dan Event* jika seluruh kriteria selesai terpenuhi.

> **Catatan**
> Teks bertanda `huruf seperti ini` adalah kode, nama file, atau perintah yang harus diketik persis sama, termasuk huruf besar dan kecilnya.

BAB I
PENDAHULUAN

1.1 Deskripsi Modul

Perhatikan aplikasi media sosial di ponselmu. Saat kamu menekan tombol suka, angkanya langsung bertambah. Saat kamu memilih tab, daftar unggahan langsung berganti. Saat kamu mengetik komentar, jumlah karakternya ikut berubah. Semua itu terjadi tanpa memuat ulang halaman, karena JavaScript mengubah halaman sebagai tanggapan atas aksi pengguna.

Modul P1-W07 mengajakmu membuat portofolio yang bereaksi seperti itu. Modul ini terdiri atas lima Kegiatan Pembelajaran yang saling menyambung: menyimpan banyak data dalam array, menyusun data lengkap dengan object, menampilkan data ke halaman melalui DOM, menanggapi klik dan ketikan dengan event, lalu membuat daftar proyek yang dibangun dari data dan bisa disaring per kategori. Modul ditutup dengan tugas dan ketentuan pengumpulan khusus minggu ini.

Hasil minggu ini adalah **daftar keahlian dinamis dan mode gelap** di portofoliomu, ditambah penghitung karakter pesan di halaman Kontak dan halaman Proyek berisi enam proyek yang dapat disaring. Data proyek yang kamu susun minggu ini akan dipindahkan ke file JSON pada Minggu 08.

Aturan yang berlaku untuk semua minggu, seperti format laporan, standar video, tenggat, dan sanksi, tidak dibahas di modul ini. Semuanya tercantum di dokumen **Panduan dan Ketentuan Kelas Industri UBIG**.

1.2 Prasyarat

Sebelum memulai, pastikan kamu sudah memiliki:

- folder `portofolio` hasil tugas Minggu 06 yang terhubung dengan repositori GitHub dan Vercel;
- pemahaman variabel `let` dan `const`, `if`, fungsi, *arrow function*, dan *template literal* dari Minggu 06;
- kebiasaan membaca pesan *error* di Console dan menguji kode dengan beberapa masukan;
- pemahaman *selector* CSS (`#id`, `.class`, dan *selector* turunan) dari Minggu 03;

• VS Code dengan Live Server, Google Chrome, dan terminal Git Bash yang sudah berjalan.

1.3 Tujuan Pembelajaran

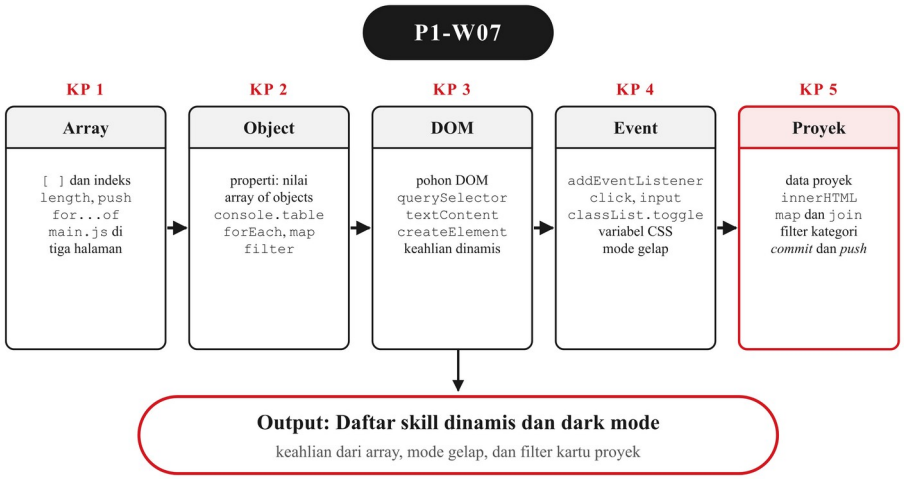
Setelah mempelajari modul ini, kamu diharapkan mampu:

1. menyimpan dan mengolah banyak data dengan array, `length`, `push`, dan perulangan `for...of`;
2. menyusun data dengan object dan array of objects, lalu mengolahnya dengan `forEach`, `map`, dan `filter`;
3. mengubah isi halaman melalui DOM dengan `querySelector`, `textContent`, `innerHTML`, `createElement`, dan `classList`;
4. menanggapi aksi pengguna dengan `addEventListener` untuk event `click` dan `input`;
5. membuat mode gelap dengan variabel CSS dan `classList.toggle`;
6. menampilkan daftar proyek dari array object dan menyaringnya berdasarkan kategori.

Tujuan tersebut membuktikan butir kompetensi **Fondasi: DOM dan Event** pada Paket 1.

1.4 Peta Konsep

Urutan materi dalam modul ini digambarkan pada Gambar 1.1. Setiap Kegiatan Pembelajaran menjadi bekal untuk kegiatan berikutnya.



Gambar 1.1 Peta konsep Modul P1-W07

1.5 Alokasi Waktu

Kegiatan minggu ini terdiri atas satu pertemuan daring 90 menit dan tugas mandiri seperti pada Tabel 1.1.

Tabel 1.1 Alokasi waktu Minggu 07

Kegiatan	Waktu	Materi
Pembuka	15 menit	Kuis P1-W06 (10 menit), lalu ulasan fungsi minggu lalu
Konsep	20 menit	Array of objects dan pohon DOM
Praktik	45 menit	Daftar keahlian dinamis dan tombol mode gelap (Kegiatan Pembelajaran 1 sampai 5)
Penutup	10 menit	Rangkuman serta penjelasan tugas filter proyek
Tugas mandiri	2 - 3 jam	Membaca ulang modul, menyelesaikan filter proyek, menulis laporan, dan merekam video

BAB II
KEGIATAN PEMBELAJARAN 1: ARRAY

2.1 Tujuan Kegiatan Pembelajaran

Setelah kegiatan ini kamu mampu:

1. menjelaskan kegunaan array dibandingkan banyak variabel terpisah;
2. membuat array dan mengambil isinya dengan indeks;
3. memakai `length` dan `push`;
4. mengulang perintah untuk setiap isi array dengan `for...of`;
5. membuat `js/main.js` dan memasangnya di ketiga halaman portofolio.

2.2 Uraian Materi

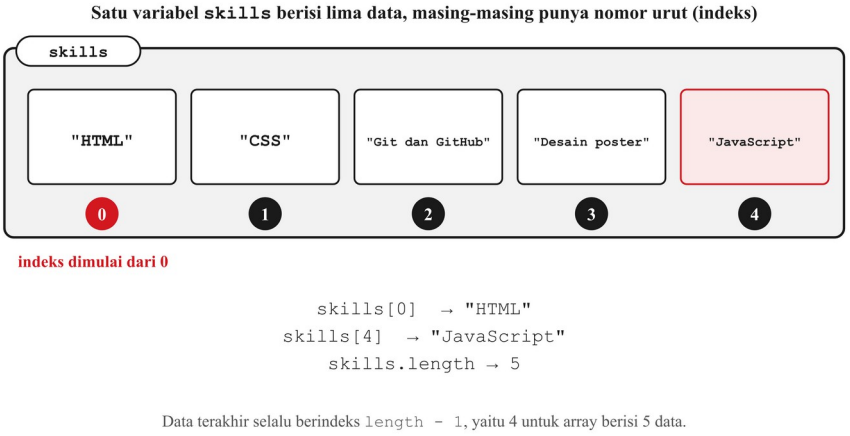
2.2.1 Satu Variabel untuk Banyak Data

Di Minggu 06 setiap variabel menyimpan satu data. Bagaimana jika datanya banyak, misalnya lima keahlian? Dengan cara lama, kamu harus membuat lima variabel seperti pada Kode 2.1.

Kode 2.1 Lima keahlian dengan lima variabel

```
const skill1 = "HTML";
const skill2 = "CSS";
const skill3 = "Git dan GitHub";
const skill4 = "Desain poster";
const skill5 = "JavaScript";
```

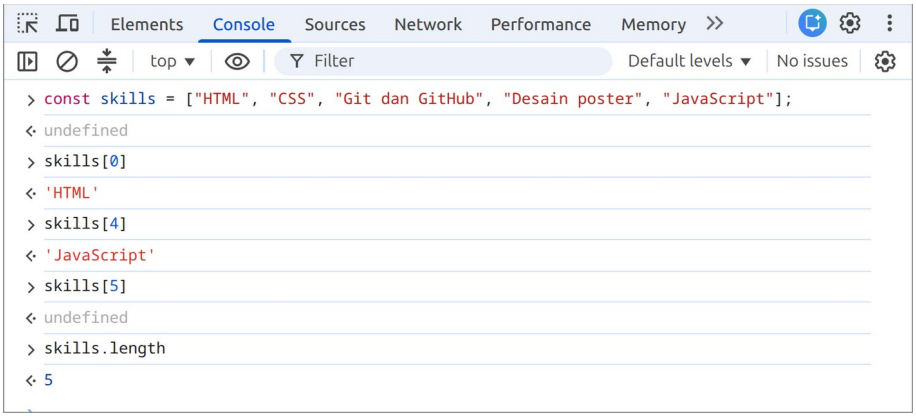
Cara ini merepotkan: menambah keahlian keenam berarti membuat variabel baru, dan menampilkan semuanya berarti menulis lima `console.log`. **Array** menyelesaikan masalah itu dengan menyimpan banyak data dalam **satu** variabel, seperti rak berlaci yang setiap lacinya bernomor. Gambaran array terlihat pada Gambar 2.1.



Gambar 2.1 Array seperti rak berlaci bernomor

2.2.2 Membuat Array dan Mengambil Isinya

Array ditulis di antara tanda kurung siku `[ ]`, dan setiap isinya dipisah koma. Isi array diambil dengan menulis nama array diikuti nomor urut di dalam kurung siku. Nomor urut ini disebut **indeks** dan selalu **dimulai dari 0**, bukan 1. Coba di Console seperti pada Gambar 2.2.



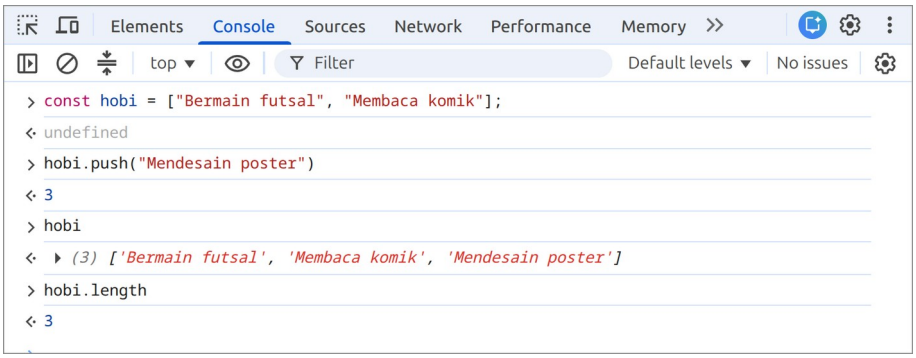
Gambar 2.2 Membuat array dan mengambil isinya dengan indeks

- `skills[0]` adalah isi pertama, `"HTML"`.
- `skills[4]` adalah isi kelima sekaligus terakhir, `"JavaScript"`.
- `skills[5]` menghasilkan `undefined` karena laci keenam tidak ada. JavaScript tidak menampilkan *error*, jadi berhati-hatilah dengan indeks.

> **Kesalahan Umum**
>
> Mengira isi pertama berindeks 1. Ingat: indeks dimulai dari 0, sehingga isi terakhir selalu berindeks `length - 1`. Jika hasilnya `undefined`, periksa kembali indeks yang kamu pakai.

2.2.3 `length` dan `push`

Array punya **properti** dan **method** bawaan. `length` menyebutkan banyaknya isi, sedangkan method `push` menambahkan isi baru di akhir array. Perhatikan Gambar 2.3.



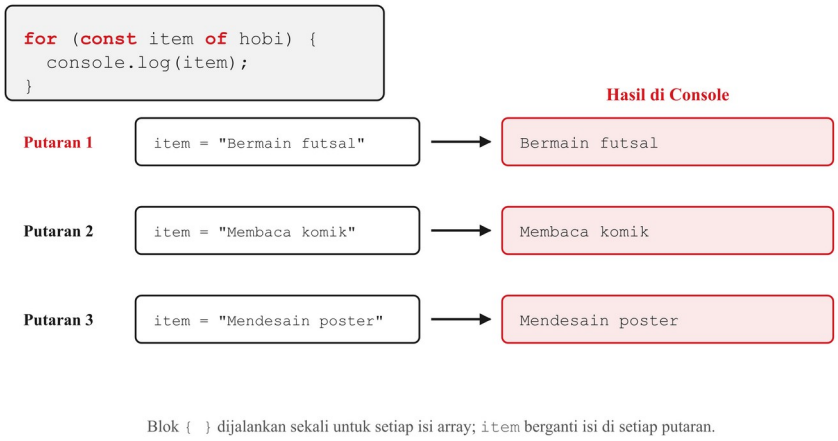
Gambar 2.3 Menambah isi array dengan `push`

`push` mengembalikan jumlah isi yang baru, yaitu `3`. Saat array ditampilkan di Console, angka di dalam kurung `(3)` juga menunjukkan banyaknya isi. Klik tanda segitiga di sebelah kirinya untuk melihat setiap isi beserta indeksnya.

> **Catatan**
>
> Array yang dibuat dengan `const` tetap boleh ditambah isinya dengan `push`. Yang tidak boleh adalah mengganti seluruh array dengan array lain, misalnya `hobi = [];`.

2.2.4 Perulangan `for...of`

Untuk menampilkan setiap isi array, kamu tidak perlu menulis `console.log` satu per satu. Gunakan **perulangan** (*loop*), yaitu perintah yang menjalankan blok kode berulang kali, sekali untuk setiap isi array. Cara kerjanya terlihat pada Gambar 2.4.



Gambar 2.4 Cara kerja perulangan `for...of`

Contoh lengkapnya ada pada Kode 2.2. Kata `item` adalah nama variabel yang kamu pilih sendiri; di setiap putaran isinya diganti dengan isi array berikutnya.

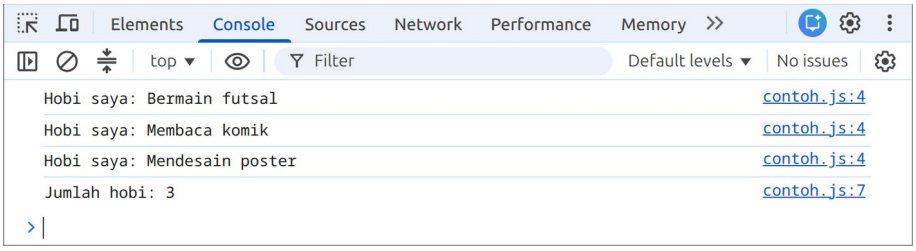
Kode 2.2 Menampilkan setiap hobi dengan `for...of`

```
const hobi = ["Bermain futsal", "Membaca komik", "Mendesain
poster"];

for (const item of hobi) {
  console.log(`Hobi saya: ${item}`);
}

console.log(`Jumlah hobi: ${hobi.length}`);
```

Hasilnya terlihat pada Gambar 2.5: blok di dalam `{ }` dijalankan tiga kali karena array `hobi` berisi tiga data.



Gambar 2.5 Hasil perulangan `for...of` di Console

2.3 Praktik: Memulai `main.js`

2.3.1 Tahap 1: Melepas `latihan.js` dan `logika.js`

Latihan Minggu 06 hanya untuk belajar dan membuat Console website *live* penuh pesan latihan. Buka `index.html`, lalu **hapus** dua baris `script` pada Kode 2.3. File `latihan.js` dan `logika.js` tetap disimpan di folder `js` sebagai arsip latihanmu.

Kode 2.3 Dua baris yang dihapus dari `index.html`

```
<script src="js/latihan.js" defer></script>
<script src="js/logika.js" defer></script>
```

2.3.2 Tahap 2: Membuat `main.js` di Ketiga Halaman

Buat file baru `js/main.js`. File inilah yang akan berisi semua JavaScript halaman portofolio. Pasang di **ketiga** halaman (`index.html`, `proyek.html`, dan `kontak.html`) tepat di bawah `link` ke `style.css`, seperti pada Kode 2.4.

Kode 2.4 Bagian `head` `index.html` dengan `main.js`

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Andi Pratama | Beranda</title>
  <link rel="icon" href="img/foto-profil.png">
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com"
crossorigin>
  <link href="https://fonts.googleapis.com/css2?
family=Inter:wght@400;600&family=Poppins:wght@600;700&display=swap"
rel="stylesheet">
  <link rel="stylesheet" href="css/style.css">
  <script src="js/main.js" defer></script>
</head>
```

2.3.3 Tahap 3: Array Keahlian

Ketik isi pertama `main.js` pada Kode 2.5, lalu simpan.

Kode 2.5 Isi pertama `js/main.js`: array keahlian

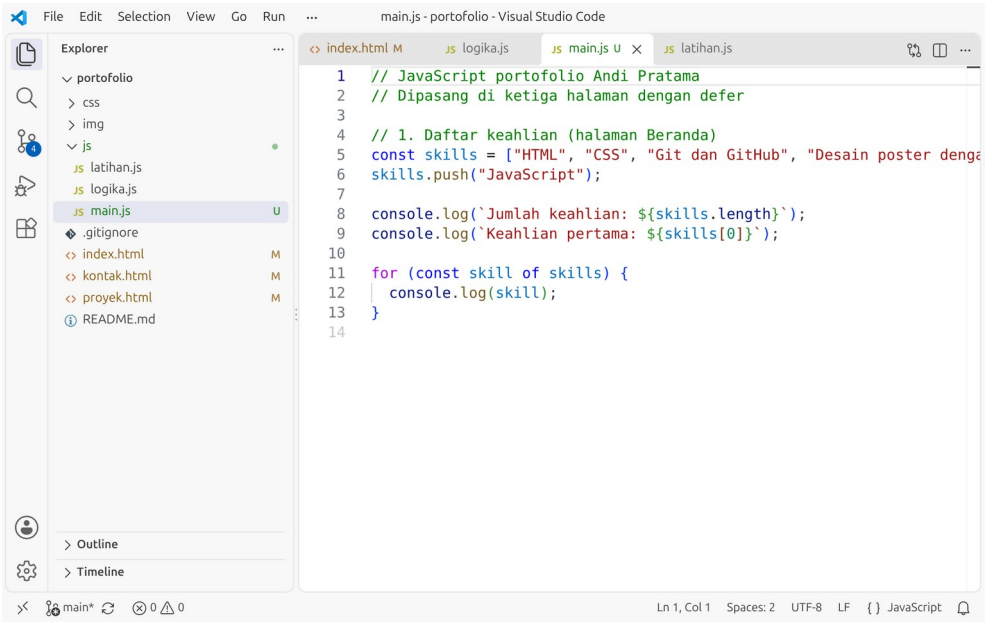
```
// JavaScript portofolio Andi Pratama
// Dipasang di ketiga halaman dengan defer

// 1. Daftar keahlian (halaman Beranda)
const skills = ["HTML", "CSS", "Git dan GitHub", "Desain poster
dengan Canva"];
skills.push("JavaScript");

console.log(`Jumlah keahlian: ${skills.length}`);
console.log(`Keahlian pertama: ${skills[0]}`);

for (const skill of skills) {
  console.log(skill);
}
```

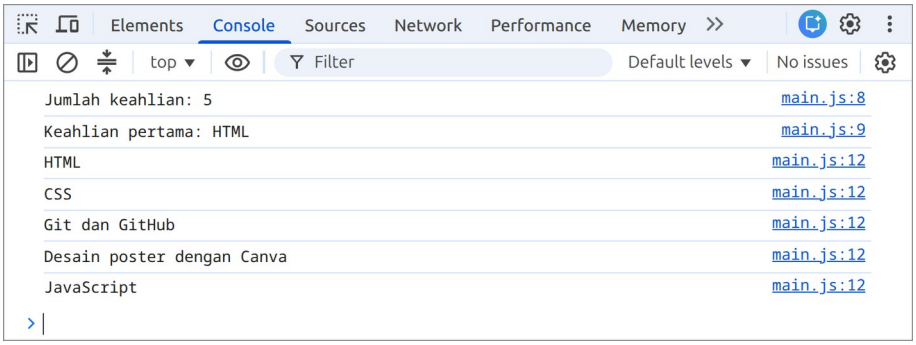
Tampilan VS Code setelah file dibuat terlihat pada Gambar 2.6.



Gambar 2.6 File `main.js` di VS Code

Buka `index.html` dengan Live Server, tekan **Ctrl+Shift+J**, lalu muat ulang. Console menampilkan jumlah keahlian, keahlian pertama, dan setiap keahlian dari perulangan seperti pada Gambar 2.7. Buka juga halaman Proyek dan Kontak; Console di

kedua halaman itu menampilkan pesan yang sama karena `main.js` terpasang di ketiganya.



Gambar 2.7 Hasil array keahlian di Console

2.4 Rangkuman

- Array menyimpan banyak data dalam satu variabel: `["HTML", "CSS"]`.
- Isi array diambil dengan indeks yang dimulai dari 0; isi terakhir berindeks `length - 1`.
- `length` menyebutkan banyaknya isi; `push` menambah isi di akhir array.
- `for (const item of array) { ... }` menjalankan blok sekali untuk setiap isi array.
- Semua JavaScript halaman portofolio ditulis di `js/main.js` yang dipasang di ketiga halaman dengan `defer`.

2.5 Latihan

1. Mengapa array lebih baik daripada lima variabel terpisah untuk menyimpan lima keahlian?
2. Diketahui `const mapel = ["Matematika", "Bahasa", "Informatika"];`. Tuliskan cara mengambil `"Informatika"` dan sebutkan nilai `mapel.length`.
3. Apa hasil `mapel[3]` pada soal nomor 2, dan mengapa?
4. Tuliskan perintah untuk menambah `"Fisika"` ke array `mapel`.
5. Tuliskan perulangan `for...of` yang menampilkan `Saya belajar Matematika`, `Saya belajar Bahasa`, dan seterusnya untuk setiap isi `mapel`.

BAB III
KEGIATAN PEMBELAJARAN 2: OBJECT DAN ARRAY OF OBJECTS

3.1 Tujuan Kegiatan Pembelajaran

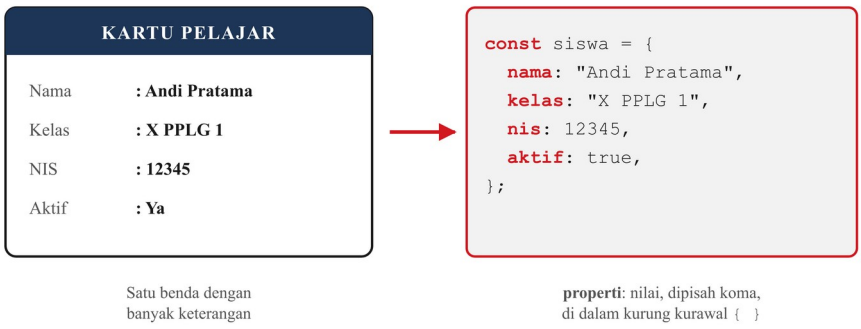
Setelah kegiatan ini kamu mampu:

1. membuat object dengan beberapa properti;
2. membaca dan mengubah properti dengan tanda titik;
3. menyusun data dalam array of objects dan menampilkannya dengan `console.table`;
4. memakai `forEach`, `map`, dan `filter` untuk mengolah array;
5. mengubah array keahlian menjadi array of objects berisi nama dan level.

3.2 Uraian Materi

3.2.1 Object: Satu Data dengan Banyak Keterangan

Array cocok untuk daftar, tetapi kadang satu data punya banyak keterangan. Kartu pelajar misalnya berisi nama, kelas, NIS, dan status aktif. Data seperti itu disimpan dalam **object**: kumpulan pasangan **properti** dan nilai di dalam kurung kurawal `{ }`, seperti pada Gambar 3.1.



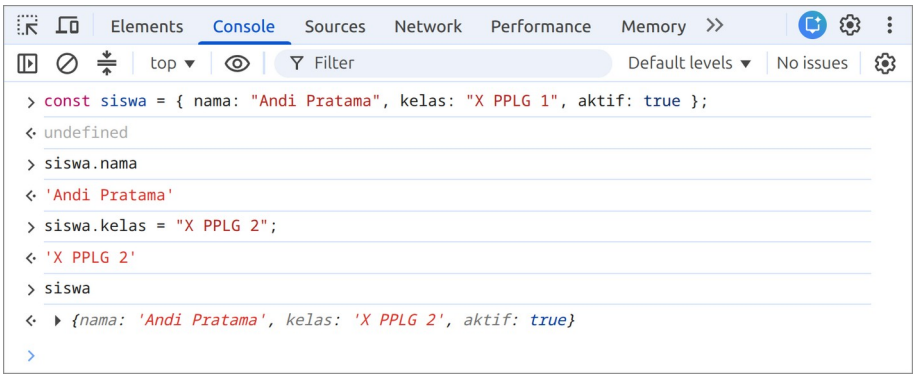
Gambar 3.1 Object seperti kartu pelajar

- Setiap baris berbentuk `properti: nilai`, dipisah koma.
- Nama properti mengikuti aturan nama variabel dan ditulis tanpa tanda kutip.

• Nilainya boleh bertipe apa saja: *string*, *number*, *boolean*, bahkan array.

3.2.2 Membaca dan Mengubah Properti

Nilai sebuah properti dibaca dengan **tanda titik**: nama object, titik, lalu nama properti. Cara yang sama dipakai untuk mengganti nilainya. Coba di Console seperti pada Gambar 3.2.

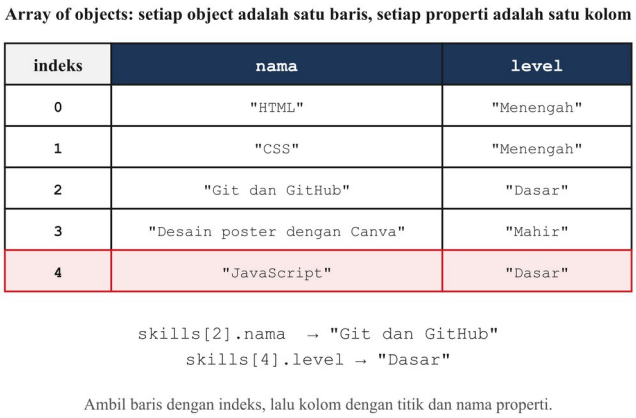


Gambar 3.2 Membaca dan mengubah properti object

`siswa.nama` menghasilkan `'Andi Pratama'`. Setelah `siswa.kelas = "X PPLG 2";`, isi object yang ditampilkan Console ikut berubah. Kamu sebenarnya sudah memakai tanda titik ini sejak Minggu 06: `console.log` berarti properti `log` milik object `console`, dan `skills.length` berarti properti `length` milik array `skills`.

3.2.3 Array of Objects

Array dan object paling sering dipakai bersama: sebuah array yang setiap isinya adalah object dengan properti yang sama. Bentuk ini disebut **array of objects** dan bisa dibayangkan seperti tabel, seperti pada Gambar 3.3.

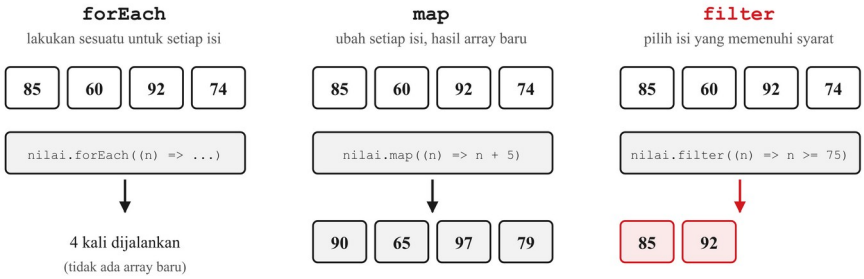


Gambar 3.3 Array of objects seperti tabel

Isi tabel dibaca dengan menggabungkan indeks dan tanda titik: `skills[2].nama` berarti "ambil baris berindeks 2, lalu kolom `nama`". Untuk melihat seluruh tabel sekaligus, Console menyediakan perintah `console.table`, yang dipakai di praktik kegiatan ini.

3.2.4 `forEach`, `map`, dan `filter`

Selain `for...of`, array punya tiga method yang sangat sering dipakai. Ketiganya menerima *arrow function* yang dijalankan untuk setiap isi array, seperti pada Gambar 3.4.



Gambar 3.4 Perbedaan `forEach`, `map`, dan `filter`

Tabel 3.1 Tiga method array

No	Method	Kegunaan	Hasil
1	`forEach`	Melakukan sesuatu untuk setiap isi, mirip `for...of`	Tidak ada (hanya menjalankan)
2	`map`	Mengubah setiap isi menjadi bentuk baru	Array baru dengan jumlah isi yang sama
3	`filter`	Memilih isi yang memenuhi syarat `true`	Array baru, isinya bisa lebih sedikit

Contoh ketiganya pada array nilai terlihat pada Kode 3.1, dan hasilnya pada Gambar 3.5.

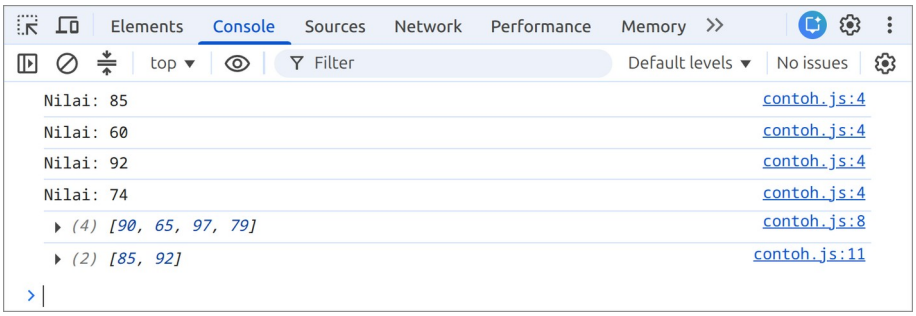
Kode 3.1 Contoh `forEach`, `map`, dan `filter`

```
const nilai = [85, 60, 92, 74];

nilai.forEach((n) => {
  console.log(`Nilai: ${n}`);
});

const nilaiPlus5 = nilai.map((n) => n + 5);
console.log(nilaiPlus5);

const lulus = nilai.filter((n) => n >= 75);
console.log(lulus);
```



Gambar 3.5 Hasil `forEach`, `map`, dan `filter` di Console

Perhatikan bahwa array `nilai` sendiri tidak berubah. `map` dan `filter` selalu membuat **array baru** yang disimpan di variabel lain, yaitu `nilaiPlus5` dan `lulus`.

Coba Sendiri

Di Console, ketik `[85, 60, 92, 74].filter((n) => n < 75)` lalu `[85, 60, 92, 74].map((n) => n * 2)`. Tebak dulu hasilnya sebelum menekan Enter.

3.3 Praktik: Keahlian sebagai Array of Objects

3.3.1 Tahap 1: Menambahkan Level Keahlian

Setiap keahlian kini diberi level: Dasar, Menengah, atau Mahir. Ganti array `skills` di `main.js` dengan array of objects pada Kode 3.2. Perhatikan bahwa `push` kini menambahkan sebuah object.

Kode 3.2 Array keahlian sebagai array of objects

```
const skills = [
  { nama: "HTML", level: "Menengah" },
  { nama: "CSS", level: "Menengah" },
  { nama: "Git dan GitHub", level: "Dasar" },
  { nama: "Desain poster dengan Canva", level: "Mahir" },
];
skills.push({ nama: "JavaScript", level: "Dasar" });
```

3.3.2 Tahap 2: Memeriksa Data di Console

Ganti seluruh `console.log` dan perulangan dari Kegiatan Pembelajaran 1 dengan kode pada Kode 3.3. Isi lengkap `main.js` sekarang terlihat pada Kode 3.4.

Kode 3.3 Menampilkan tabel dan keahlian yang sedang dipelajari

```
console.table(skills);

const sedangDipelajari = skills.filter((skill) => skill.level ===
"Dasar");
console.log(sedangDipelajari.map((skill) => skill.nama));
```

Kode 3.4 Isi `js/main.js` setelah Kegiatan Pembelajaran 2

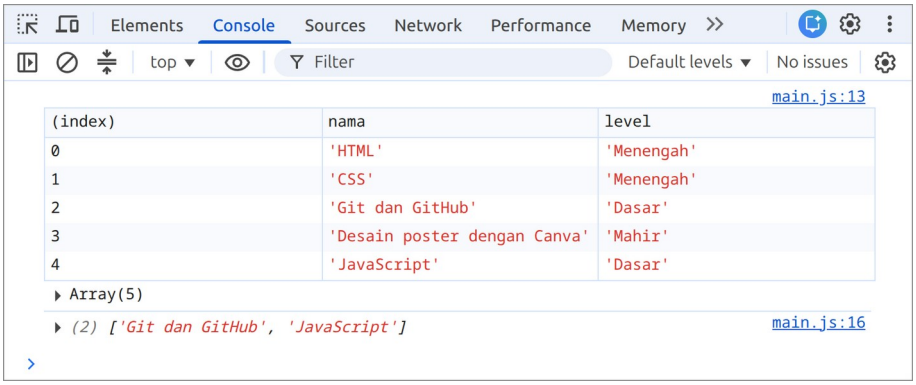
```
// JavaScript portofolio Andi Pratama
// Dipasang di ketiga halaman dengan defer

// 1. Daftar keahlian (halaman Beranda)
const skills = [
  { nama: "HTML", level: "Menengah" },
  { nama: "CSS", level: "Menengah" },
  { nama: "Git dan GitHub", level: "Dasar" },
  { nama: "Desain poster dengan Canva", level: "Mahir" },
];
skills.push({ nama: "JavaScript", level: "Dasar" });

console.table(skills);

const sedangDipelajari = skills.filter((skill) => skill.level ===
"Dasar");
console.log(sedangDipelajari.map((skill) => skill.nama));
```

Simpan dan muat ulang halaman. `console.table` menampilkan seluruh keahlian sebagai tabel, lalu baris berikutnya menampilkan nama keahlian yang levelnya Dasar, seperti pada Gambar 3.6.



Gambar 3.6 Array keahlian sebagai tabel di Console

- `filter` memilih object yang `level`-nya `"Dasar"`, sehingga hasilnya array berisi dua object.
- `map` mengubah setiap object itu menjadi namanya saja, sehingga hasil akhirnya `['Git dan GitHub', 'JavaScript']`.

3.4 Rangkuman

- Object menyimpan satu data dengan banyak keterangan: `{ nama: "HTML", level: "Menengah" }`.

- Properti dibaca dan diubah dengan tanda titik, misalnya `siswa.nama`.
- Array of objects menyimpan banyak data sejenis seperti tabel; `console.table` menampilkannya sebagai tabel.
- `forEach` menjalankan sesuatu untuk setiap isi, `map` membuat array baru hasil ubahan, dan `filter` membuat array baru berisi yang memenuhi syarat.

3.5 Latihan

1. Buatlah object `buku` dengan properti `judul`, `penulis`, dan `halaman`.
2. Dari object `buku` pada soal nomor 1, tuliskan cara membaca judulnya dan cara mengganti jumlah halamannya menjadi 250.
3. Apa perbedaan array of objects dengan object biasa? Berikan contoh data sekolah untuk masing-masing.
4. Diketahui `const nilai = [70, 88, 95, 64];`. Tuliskan kode `filter` yang menghasilkan nilai di atas 80 dan sebutkan hasilnya.
5. Pada array `skills`, apa hasil `skills.map((skill) => skill.level)`?

BAB IV
KEGIATAN PEMBELAJARAN 3: DOM

4.1 Tujuan Kegiatan Pembelajaran

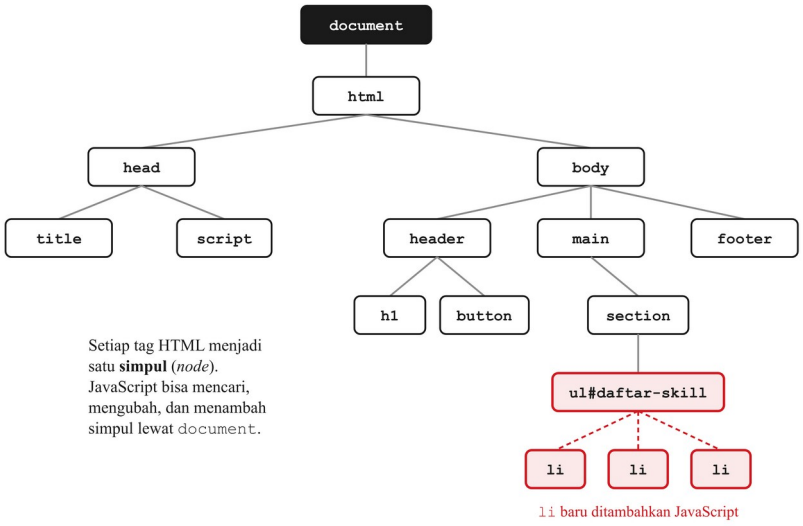
Setelah kegiatan ini kamu mampu:

1. menjelaskan DOM sebagai pohon elemen halaman;
2. mengambil elemen dengan `querySelector` dan memahami hasil `null`;
3. mengubah isi elemen dengan `textContent` dan `innerHTML`;
4. membuat elemen baru dengan `createElement` dan menempelkannya dengan `appendChild`;
5. menampilkan daftar keahlian di Beranda dari array keahlian.

4.2 Uraian Materi

4.2.1 Pohon DOM

Saat membuka halaman, *browser* membaca HTML lalu menyusunnya menjadi struktur di memori yang disebut **DOM** (*Document Object Model*). Setiap tag menjadi satu **simpul** (*node*) yang bercabang seperti pohon keluarga, seperti pada Gambar 4.1. Pangkal pohonnya adalah object `document`.



Gambar 4.1 Pohon DOM halaman Beranda

JavaScript tidak mengubah file HTML-mu. Yang diubah adalah DOM, yaitu salinan halaman di *browser*. Begitu DOM berubah, tampilan halaman langsung ikut berubah tanpa dimuat ulang. Saat halaman dimuat ulang, DOM disusun lagi dari file HTML, lalu JavaScript menjalankan perubahannya kembali.

4.2.2 Mengambil Elemen dengan `querySelector`

Sebelum diubah, elemen harus diambil dulu dengan `document.querySelector("selector")`. *Selector*-nya sama persis dengan *selector* CSS dari Minggu 03: `h1`, `.kartu`, `#daftar-skill`, atau `.hero h2`. Hasilnya adalah elemen **pertama** yang cocok. Coba di Console halaman Beranda seperti pada Gambar 4.2.



Gambar 4.2 Mengambil elemen dengan `querySelector`

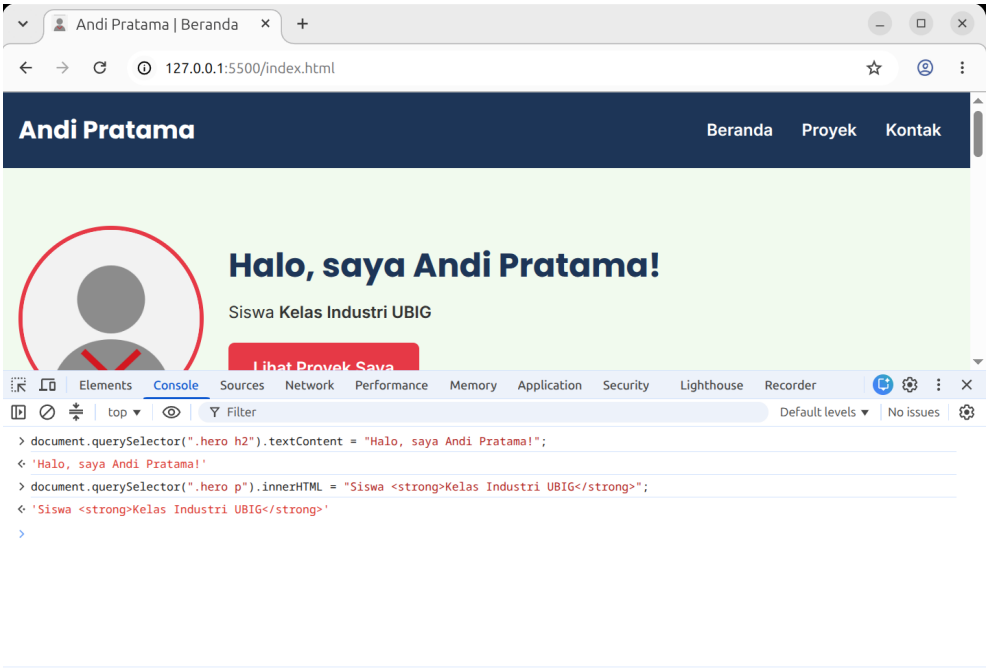
Elemen yang ditemukan ditampilkan sebagai tag HTML; arahkan *mouse* ke hasilnya dan elemen itu ditandai biru di halaman. Jika tidak ada elemen yang cocok, hasilnya `null`, yang berarti "kosong, tidak ada apa-apa". Hasil `null` ini akan menjadi penyebab *error* di Tahap 4 praktik.

4.2.3 Mengubah Isi: `textContent` dan `innerHTML`

Setelah elemen diambil, isinya bisa diganti dengan dua properti:

- `textContent` mengganti isi dengan **teks biasa**. Tanda < dan > ditampilkan apa adanya, bukan dibaca sebagai tag.
- `innerHTML` mengganti isi dengan **HTML**, sehingga tag seperti `<strong>` ikut dibaca *browser*.

Ketik dua perintah di Console halaman Beranda. Judul dan paragraf di bagian atas halaman langsung berubah seperti pada Gambar 4.3.



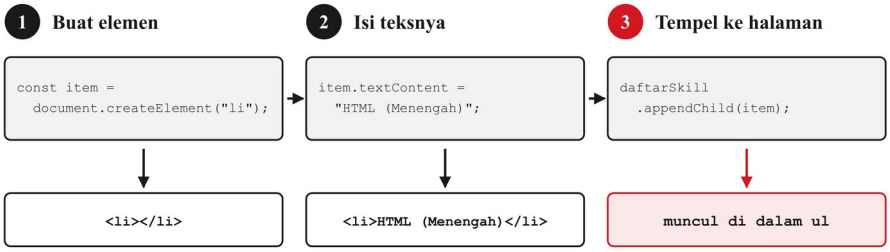
Gambar 4.3 Mengubah judul dan paragraf dari Console

Muat ulang halaman, dan judul serta paragraf kembali seperti semula. Perubahan dari Console hanya terjadi di DOM, bukan di file `index.html`.

> **Perhatian**
>
> Gunakan `innerHTML` hanya untuk data milikmu sendiri, seperti data proyek di Kegiatan Pembelajaran 5. Jangan memasukkan ketikan pengunjung ke `innerHTML`, karena ketikan itu bisa berisi tag atau kode yang ikut dijalankan. Untuk teks dari pengunjung, selalu pakai `textContent`.

4.2.4 Membuat Elemen Baru: `createElement` dan `appendChild`

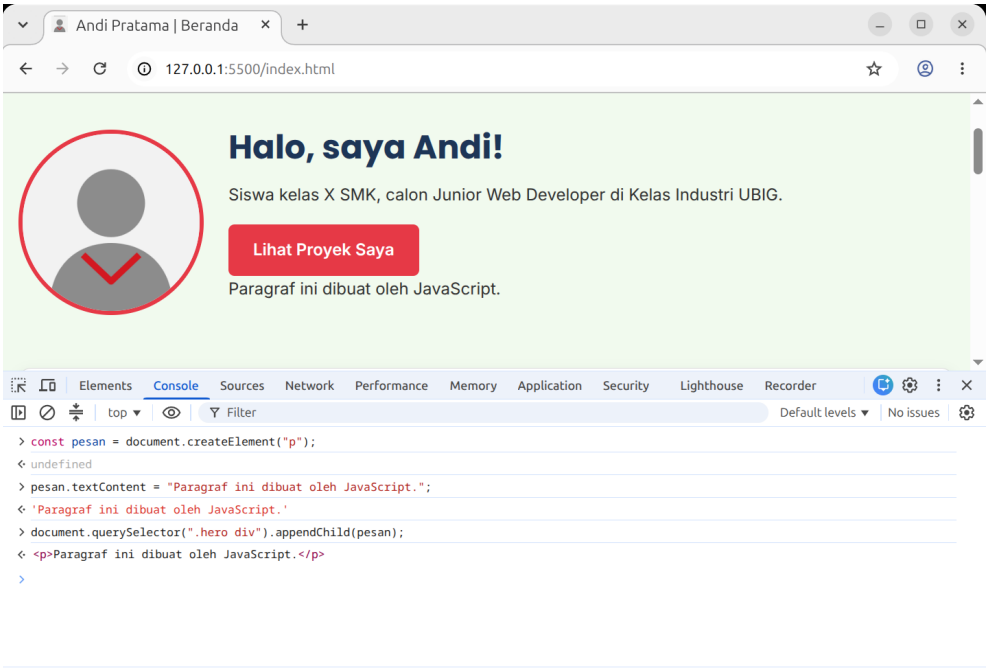
Untuk menambahkan elemen yang belum ada di HTML, lakukan tiga langkah pada Gambar 4.4: buat elemen, isi teksnya, lalu tempelkan ke elemen induk di halaman.



Sebelum langkah 3, elemen baru hanya ada di memori dan belum terlihat di halaman.

Gambar 4.4 Tiga langkah menambahkan elemen baru

Coba tambahkan satu paragraf ke bagian hero dari Console. Paragraf baru muncul tepat di bawah tombol, seperti pada Gambar 4.5.



Gambar 4.5 Paragraf baru ditambahkan dari Console

4.2.5 Mengatur Class dengan `classList`

Selain isi, JavaScript juga bisa menambah atau membuang class sebuah elemen melalui `classList`. Karena tampilan diatur CSS lewat class, mengganti class berarti mengganti tampilan. Empat method yang dipakai minggu ini tercantum pada Tabel 4.1.

Tabel 4.1 Method `classList`

No	Method	Kegunaan
1	`classList.add("gelap")`	Menambahkan class `gelap`
2	`classList.remove("gelap")`	Membuang class `gelap`
3	`classList.toggle("gelap")`	Menambahkan jika belum ada, membuang jika sudah ada (seperti saklar)
4	`classList.contains("gelap")`	Memeriksa apakah class `gelap` ada; hasilnya `true` atau `false`

Method ini dipakai untuk tombol mode gelap di Kegiatan Pembelajaran 4.

4.3 Praktik: Daftar Keahlian Dinamis

4.3.1 Tahap 1: Mengosongkan Daftar di HTML

Daftar keahlian kini akan dibuat JavaScript, jadi isinya di HTML dihapus. Di `index.html`, ganti `ul` beserta keempat `li` di kartu Keahlian dengan dua elemen kosong pada Kode 4.1. Atribut `id` membuat keduanya mudah diambil dengan `querySelector`.

Kode 4.1 Kartu Keahlian di `index.html` setelah dikosongkan

```
<section class="kartu">
  <h2>Keahlian</h2>
  <ul id="daftar-skill"></ul>
  <p id="jumlah-skill"></p>
</section>
```

4.3.2 Tahap 2: Menampilkan Keahlian dengan `createElement`

Di `main.js`, hapus kode bagian 1 dari Kegiatan Pembelajaran 2 **setelah** array `skills` (yaitu `console.table` sampai `console.log` terakhir), lalu tambahkan kode pada Kode 4.2.

Kode 4.2 Menampilkan setiap keahlian ke dalam daftar

```
const daftarSkill = document.querySelector("#daftar-skill");
const jumlahSkill = document.querySelector("#jumlah-skill");

skills.forEach((skill) => {
  const item = document.createElement("li");
  item.textContent = `${skill.nama} (${skill.level})`;
  daftarSkill.appendChild(item);
});
jumlahSkill.textContent = `Ada ${skills.length} keahlian yang sedang
saya kembangkan.`;
```

- `daftarSkill` dan `jumlahSkill` menyimpan elemen hasil `querySelector`, supaya tidak perlu dicari berulang kali.
- `forEach` menjalankan tiga langkah `createElement`, `textContent`, dan `appendChild` untuk setiap keahlian.
- Baris terakhir menulis jumlah keahlian dari `skills.length`, sehingga angkanya otomatis bertambah jika keahlian ditambah.

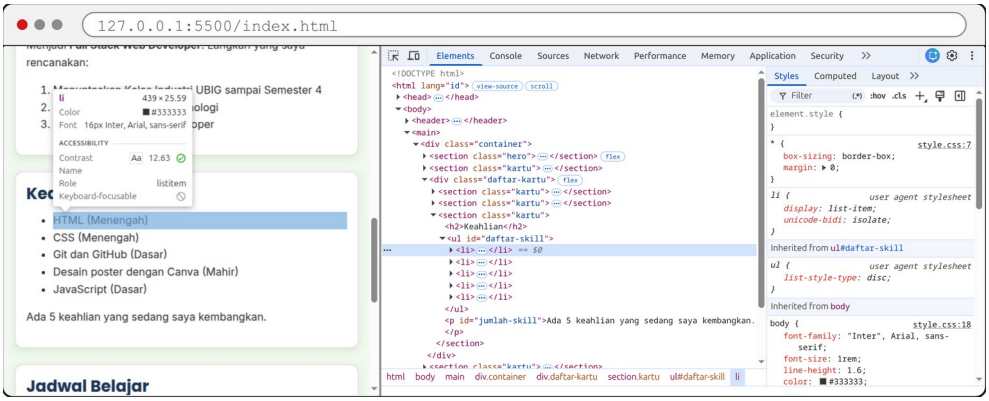
Simpan, lalu lihat kartu Keahlian di Beranda. Kelima keahlian tampil lengkap dengan levelnya seperti pada Gambar 4.6, padahal di HTML daftar itu kosong.



Gambar 4.6 Daftar keahlian dibuat dari array

4.3.3 Tahap 3: Membuktikan Daftar Berasal dari Array

Klik kanan salah satu keahlian, lalu pilih **Inspect**. Panel Elements menampilkan DOM yang sedang berlaku: lima `li` sudah ada di dalam `ul#daftar-skill`, seperti pada Gambar 4.7. Sebaliknya, jika kamu membuka file `index.html` di VS Code atau menekan **Ctrl+U** (View Page Source), `ul` itu tetap kosong. Inilah bukti bahwa daftar keahlian berasal dari array, sesuai kriteria minggu ini.

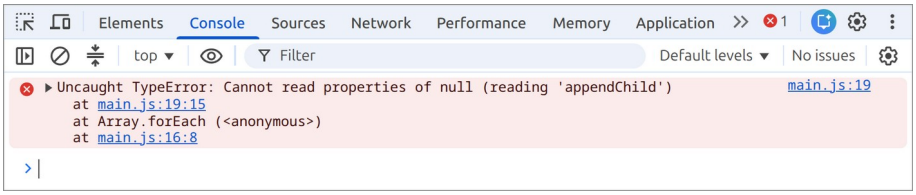


Gambar 4.7 Elemen `li` hasil JavaScript di panel Elements

> **Coba Sendiri**
>
> Tambahkan satu object keahlian lagi ke array `skills`, misalnya `{ nama: "Mengetik cepat", level: "Mahir" }`, lalu simpan. Daftar dan kalimat jumlah keahlian langsung bertambah tanpa menyentuh HTML. Hapus kembali setelah mencoba, atau biarkan jika memang keahlianmu.

4.3.4 Tahap 4: Memperbaiki Error di Halaman Lain

Sekarang buka halaman **Proyek** dan lihat Console. Muncul *error* merah seperti pada Gambar 4.8.



Gambar 4.8 `TypeError` karena elemen tidak ada di halaman Proyek

`main.js` juga dijalankan di halaman Proyek, padahal halaman itu tidak punya `#daftar-skill`. Akibatnya `daftarSkill` berisi `null`, dan `null` tidak punya method `appendChild`. Pesan `Cannot read properties of null` berarti "tidak bisa membaca sesuatu dari nilai `null`". Perbaikannya adalah menjalankan kode keahlian **hanya jika** elemennya ada, dengan `if` dari Minggu 06. Bungkus kode `forEach` dan `jumlahSkill` dengan `if (daftarSkill) { ... }` seperti pada Kode 4.3.

Kode 4.3 Isi `js/main.js` setelah Kegiatan Pembelajaran 3

```
// JavaScript portofolio Andi Pratama
// Dipasang di ketiga halaman dengan defer

// 1. Daftar keahlian (halaman Beranda)
const skills = [
  { nama: "HTML", level: "Menengah" },
  { nama: "CSS", level: "Menengah" },
  { nama: "Git dan GitHub", level: "Dasar" },
  { nama: "Desain poster dengan Canva", level: "Mahir" },
];
skills.push({ nama: "JavaScript", level: "Dasar" });

const daftarSkill = document.querySelector("#daftar-skill");
const jumlahSkill = document.querySelector("#jumlah-skill");

if (daftarSkill) {
  skills.forEach((skill) => {
    const item = document.createElement("li");
    item.textContent = `${skill.nama} (${skill.level})`;
    daftarSkill.appendChild(item);
  });
  jumlahSkill.textContent = `Ada ${skills.length} keahlian yang
sedang saya kembangkan.`;
}
```

Di dalam kondisi `if`, elemen yang ditemukan dianggap `true`, sedangkan `null` dianggap `false`. Simpan, lalu periksa Console di halaman Proyek dan Kontak: *error*-nya hilang, dan daftar keahlian di Beranda tetap tampil.

4.4 Rangkuman

- DOM adalah susunan halaman di *browser* berbentuk pohon; JavaScript mengubah DOM, bukan file HTML.
- `document.querySelector("selector")` mengambil elemen pertama yang cocok, atau `null` jika tidak ada.

- `textContent` mengganti isi dengan teks biasa; `innerHTML` mengganti isi dengan HTML dan hanya dipakai untuk data milik sendiri.
- Elemen baru dibuat dengan `createElement`, diisi, lalu ditempel dengan `appendChild`.
- `classList` menambah, membuang, membalik, dan memeriksa class; kode untuk satu halaman dibungkus `if (elemen)` agar tidak *error* di halaman lain.

4.5 Latihan

1. Apa yang dimaksud DOM, dan apa bedanya dengan file HTML?
2. Tuliskan kode untuk mengambil elemen `footer` lalu mengganti teksnya menjadi `Dibuat oleh Andi`.
3. Apa perbedaan `textContent` dan `innerHTML`? Kapan `innerHTML` sebaiknya tidak dipakai?
4. Tuliskan tiga baris kode untuk menambahkan `li` berisi `Bermain gitar` ke `ul` dengan `id="daftar-hobi"`.
5. Console menampilkan `Cannot read properties of null (reading 'appendChild')`. Jelaskan penyebab dan cara memperbaikinya.

BAB V
KEGIATAN PEMBELAJARAN 4: EVENT DAN MODE GELAP

5.1 Tujuan Kegiatan Pembelajaran

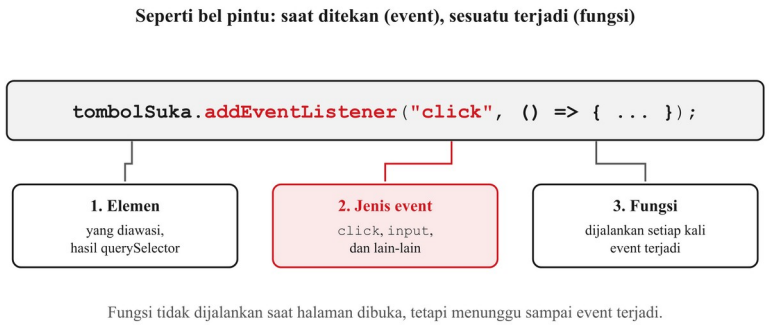
Setelah kegiatan ini kamu mampu:

1. menjelaskan event dan menulis `addEventListener`;
2. menanggapi event `click` dan `input`;
3. memakai variabel CSS untuk menyimpan warna tema;
4. membuat tombol mode gelap dengan `classList.toggle` yang berlaku di semua bagian halaman;
5. membuat penghitung karakter pesan di halaman Kontak.

5.2 Uraian Materi

5.2.1 Event dan `addEventListener`

Event adalah kejadian di halaman, misalnya tombol diklik, teks diketik, atau halaman digulir. JavaScript bisa "mendengarkan" event pada sebuah elemen, lalu menjalankan fungsi setiap kali event itu terjadi. Caranya adalah method `addEventListener` dengan tiga bagian pada Gambar 5.1.



Gambar 5.1 Bagian-bagian `addEventListener`

Fungsi di bagian ketiga biasanya ditulis sebagai *arrow function* tanpa parameter. Fungsi itu tidak dijalankan saat halaman dibuka, tetapi menunggu sampai event terjadi, dan bisa dijalankan berkali-kali.

5.2.2 Event `click`

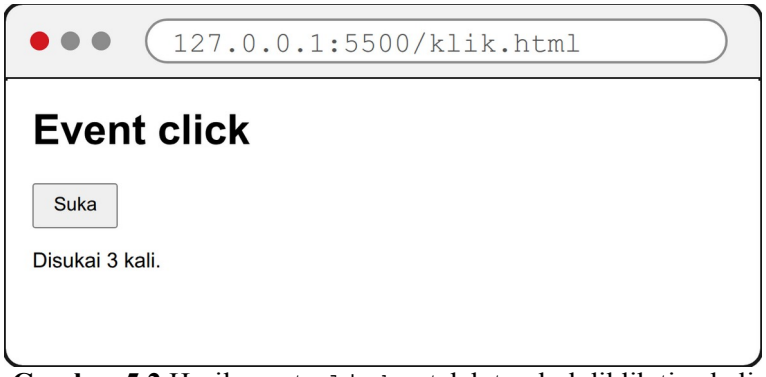
Event `click` terjadi saat elemen diklik. Contoh pada Kode 5.1 menambah angka setiap kali tombol **Suka** diklik.

Kode 5.1 Tombol suka dengan event `click`

```
const tombolSuka = document.querySelector("#tombol-suka");
const jumlahSuka = document.querySelector("#jumlah-suka");
let suka = 0;

tombolSuka.addEventListener("click", () => {
  suka = suka + 1;
  jumlahSuka.textContent = `Disukai ${suka} kali.`;
});
```

Variabel `suka` memakai `let` karena isinya bertambah. Setelah tombol diklik tiga kali, paragraf menampilkan `Disukai 3 kali.` seperti pada Gambar 5.2.



Gambar 5.2 Hasil event `click` setelah tombol diklik tiga kali

5.2.3 Event `input`

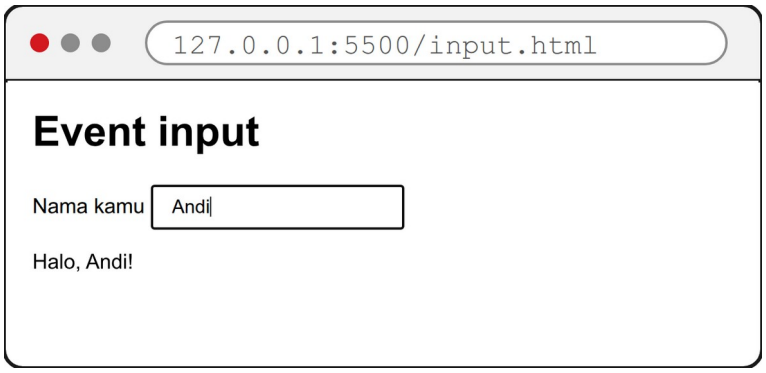
Event `input` terjadi setiap kali isi kotak isian berubah, yaitu di setiap huruf yang diketik atau dihapus. Isi kotaknya dibaca dari properti `value`. Contoh pada Kode 5.2 menyapa nama yang sedang diketik.

Kode 5.2 Sapaan otomatis dengan event `input`

```
const kotakNama = document.querySelector("#nama");
const sapaan = document.querySelector("#sapaan");

kotakNama.addEventListener("input", () => {
  sapaan.textContent = `Halo, ${kotakNama.value}!`;
});
```

Saat `Andi` diketik, paragraf langsung berubah menjadi `Halo, Andi!` tanpa menekan tombol apa pun, seperti pada Gambar 5.3.



Gambar 5.3 Hasil event `input` saat nama diketik

> **Kesalahan Umum**
>
> Menulis `addEventListener("click", tampilkan())` dengan tanda kurung membuat fungsi langsung dijalankan sekali saat halaman dibuka, bukan saat diklik. Berikan fungsinya tanpa dijalankan: `() => { ... }` atau nama fungsinya saja tanpa tanda kurung. Periksa juga ejaan nama event: `click`, bukan `onclick` atau `Click`.

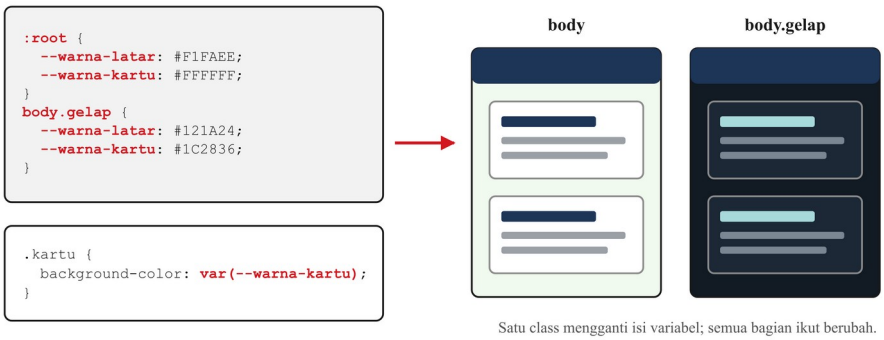
5.2.4 Variabel CSS untuk Tema

Mode gelap berarti mengganti banyak warna sekaligus: latar, kartu, teks, garis, dan judul. Jika setiap warna ditulis langsung di banyak aturan CSS, mengganti tema berarti mengubah puluhan baris. Solusinya adalah **variabel CSS** (*custom property*):

- variabel dibuat dengan nama berawalan dua tanda hubung, misalnya `--warna-latar: #F1FAEE;`;
- variabel umum ditulis di dalam `:root`, yaitu elemen `html` sebagai pangkal halaman, sehingga berlaku di seluruh halaman;

• nilainya dipakai dengan `var()`, misalnya `background-color: var(--warna-latar);`.

Kekuatannya terlihat pada Gambar 5.4: jika `body` mendapat class `gelap` yang mengisi ulang variabel dengan warna gelap, semua aturan yang memakai variabel itu ikut berubah sekaligus.



Gambar 5.4 Variabel CSS untuk tema terang dan gelap

5.3 Praktik A: Tombol Mode Gelap

5.3.1 Tahap 1: Menambahkan Tombol di Header

Di **ketiga** halaman, tambahkan tombol pada Kode 5.3 tepat di bawah `</nav>`, masih di dalam `div .container .navbar`. Karena `.navbar` memakai Flexbox, tombol otomatis tersusun di samping menu.

Kode 5.3 Tombol tema di header ketiga halaman

```
      <button type="button" id="tombol-tema" class="tombol-
tema">Mode Gelap</button>
```

Lalu tambahkan bagian 10 di akhir `style.css` pada Kode 5.4. Aturan `button` dari Minggu 03 memberi latar merah, jadi aturan ini menggantinya dengan latar transparan dan garis putih.

Kode 5.4 Bagian 10 `style.css`: tombol tema

```
/* 10. Tombol tema, filter, dan label kategori */
.tombol-tema {
  padding: 0.5rem 0.9rem;
  border: 2px solid #FFFFFF;
  background-color: transparent;
  font-size: 0.9rem;
}
```

Tombol **Mode Gelap** kini tampil di header seperti pada Gambar 5.5.



Gambar 5.5 Tombol Mode Gelap di header

5.3.2 Tahap 2: Mencoba `classList.toggle`

Tambahkan bagian 2 di akhir `main.js` pada Kode 5.5. Tombol ada di ketiga halaman, jadi kode ini tidak perlu dibungkus `if`.

Kode 5.5 Bagian 2 `main.js`: tombol tema

```
// 2. Tema gelap (semua halaman)
const tombolTema = document.querySelector("#tombol-tema");

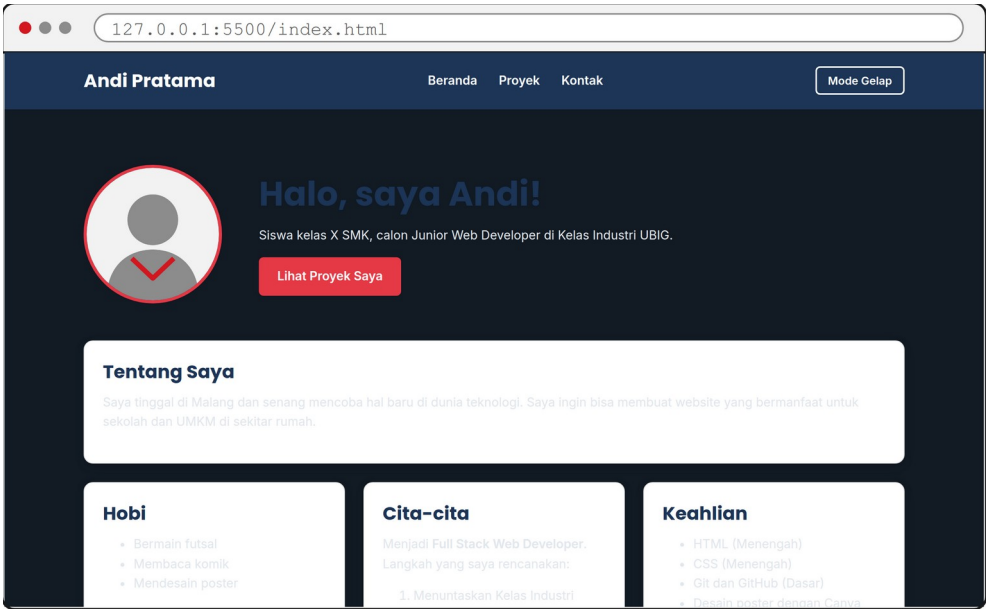
tombolTema.addEventListener("click", () => {
  document.body.classList.toggle("gelap");
});
```

Untuk mencobanya, tambahkan aturan sementara pada Kode 5.6 di akhir `style.css`, lalu klik tombolnya.

Kode 5.6 Tema gelap sederhana (sementara)

```
/* 11. Tema gelap (percobaan pertama) */
body.gelap {
  background-color: #121A24;
  color: #E3E8EE;
}
```

Latar halaman berubah gelap, tetapi kartu tetap putih, judul tetap biru tua, dan teks di kartu sulit dibaca, seperti pada Gambar 5.6. Klik lagi, dan halaman kembali terang. `toggle` bekerja, tetapi CSS-nya baru mengubah `body`. Mode gelap belum berlaku di semua bagian halaman.



Gambar 5.6 Percobaan pertama: hanya latar yang berubah

5.3.3 Tahap 3: Mengubah Warna Menjadi Variabel CSS

Hapus aturan sementara dari Tahap 2. Lalu tambahkan variabel warna di **awal bagian 2** `style.css` pada Kode 5.7.

Kode 5.7 Variabel warna tema terang di `:root`

```
:root {
  --warna-utama: #1D3557;
  --warna-judul: #1D3557;
  --warna-aksen: #E63946;
  --warna-tautan: #E63946;
  --warna-latar: #F1FAEE;
  --warna-teks: #333333;
  --warna-kartu: #FFFFFF;
  --warna-garis: #DDDDDD;
}
```

Berikutnya ganti setiap warna tetap di `style.css` dengan variabelnya seperti pada Tabel 5.1. Warna putih `#FFFFFF` untuk teks di header, menu, footer, dan tombol tetap dibiarkan, karena latar header dan footer selalu gelap di kedua tema.

Tabel 5.1 Warna tetap yang diganti variabel di `style.css`

No	Aturan dan Properti	Sebelum	Sesudah
1	`body color`	`#333333`	`var(--warna-teks)`
2	`body background-color`	`#F1FAEE`	`var(--warna-latar)`
3	`h1, h2, h3 color`	`#1D3557`	`var(--warna-judul)`
4	`a color`	`#E63946`	`var(--warna-tautan)`
5	`header`, `th`, `footer background-color`	`#1D3557`	`var(--warna-utama)`
6	`.hero img border`, `.tombol` dan `button background-color`	`#E63946`	`var(--warna-aksen)`
7	`.kartu background-color`	`#FFFFFF`	`var(--warna-kartu)`
8	`.kartu`, `article img`, `th, td`, dan `input, select, textarea border`	`#DDDDDD` atau `#CCCCCC`	`var(--warna-garis)`

Kolom isian form juga perlu mengikuti tema. Tambahkan dua baris pada Kode 5.8 ke aturan `input, select, textarea` di bagian 6.

Kode 5.8 Dua baris baru di aturan `input, select, textarea`

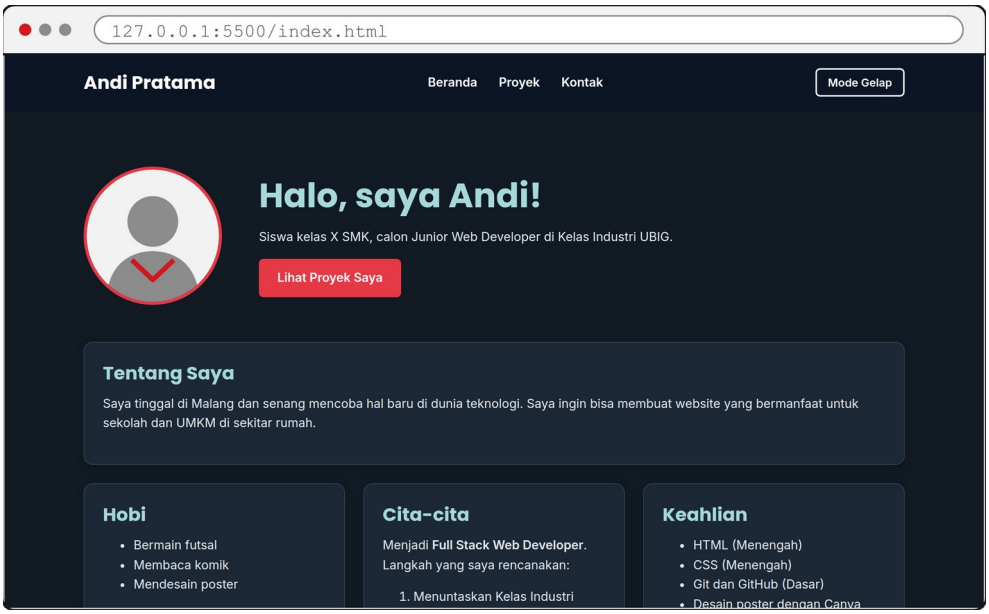
```
background-color: var(--warna-kartu);
color: var(--warna-teks);
```

Terakhir, tambahkan bagian 11 pada Kode 5.9 di akhir `style.css`. Bagian ini tidak menulis ulang aturan apa pun; ia hanya mengisi ulang variabel saat `body` memiliki class `gelap`.

Kode 5.9 Bagian 11 `style.css`: tema gelap dengan variabel

```
/* 11. Tema gelap: isi variabel diganti saat body memiliki class
gelap */
body.gelap {
  --warna-utama: #0B1526;
  --warna-judul: #A8DADC;
  --warna-tautan: #FF8A8A;
  --warna-latar: #121A24;
  --warna-teks: #E3E8EE;
  --warna-kartu: #1C2836;
  --warna-garis: #34465A;
}
```

Simpan, lalu klik tombol. Kini seluruh halaman berubah: latar, kartu, judul, teks, tabel, dan garis, seperti pada Gambar 5.7.



Gambar 5.7 Mode gelap berlaku di semua bagian halaman

> **Kesalahan Umum**
>
> Ada bagian yang tetap terang? Berarti masih ada warna tetap yang belum diganti variabel. Klik kanan bagian itu, pilih **Inspect**, lalu lihat panel Styles: properti dengan kode warna langsung (misalnya `#FFFFFF`) adalah yang perlu diganti. Pastikan juga nama variabel di `var()` sama persis dengan yang di `:root`, termasuk dua tanda hubung di depannya.

5.3.4 Tahap 4: Mengganti Teks Tombol

Saat mode gelap aktif, tombol sebaiknya bertuliskan **Mode Terang**. Ganti isi `addEventListener` di bagian 2 `main.js` dengan kode pada Kode 5.10. Setelah `toggle`, kode memeriksa class dengan `contains` dan menulis teks yang sesuai.

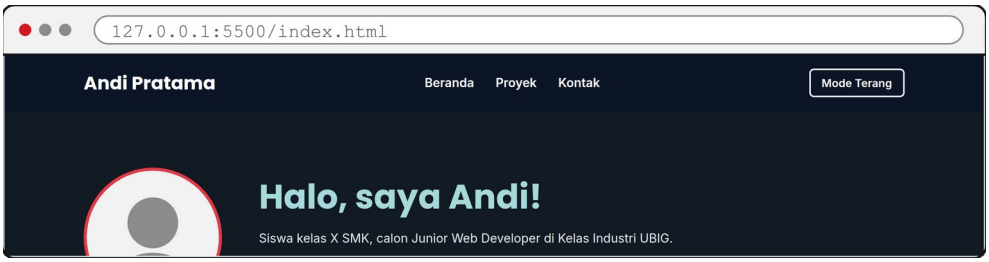
Kode 5.10 Bagian 2 `main.js` dengan teks tombol

```
// 2. Tema gelap (semua halaman)
const tombolTema = document.querySelector("#tombol-tema");

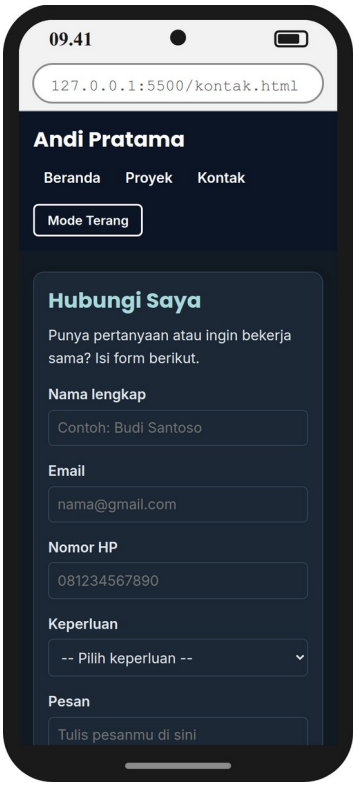
tombolTema.addEventListener("click", () => {
  document.body.classList.toggle("gelap");

  if (document.body.classList.contains("gelap")) {
    tombolTema.textContent = "Mode Terang";
  } else {
    tombolTema.textContent = "Mode Gelap";
  }
});
```

Klik tombolnya; teks berubah menjadi **Mode Terang** seperti pada Gambar 5.8. Coba juga di halaman Proyek dan Kontak, serta di mode perangkat 360 piksel seperti pada Gambar 5.9.



Gambar 5.8 Teks tombol berubah menjadi Mode Terang



Gambar 5.9 Mode gelap di halaman Kontak pada layar ponsel

> **Catatan**
>
> Jika kamu pindah halaman, tema kembali terang karena setiap halaman dimuat dari awal. Cara mengingat pilihan tema dibahas di tantangan minggu ini.

5.4 Praktik B: Penghitung Karakter Pesan

5.4.1 Tahap 1: Batas dan Tempat Penghitung

Di `kontak.html`, tambahkan atribut `maxlength="300"` pada `textarea` pesan agar pesan paling banyak 300 karakter, lalu tambahkan elemen `span` penghitung tepat di bawahnya seperti pada Kode 5.11. Atribut `maxlength` sudah kamu kenal dari Minggu 02.

Kode 5.11 Kotak pesan dan penghitung karakter di `kontak.html`

```
<textarea id="pesan" name="pesan" rows="5" cols="40" maxlength="300"
placeholder="Tulis pesanmu di sini" required></textarea><br>
<span id="hitung-karakter" class="keterangan">0/300 karakter</span>
```

Tambahkan juga aturan kecil pada Kode 5.12 di bagian 6 `style.css` agar teks penghitung sedikit lebih kecil.

Kode 5.12 Aturan `.keterangan` di bagian 6 `style.css`

```
.keterangan {
  font-size: 0.875rem;
}
```

5.4.2 Tahap 2: Menghitung dengan Event `input`

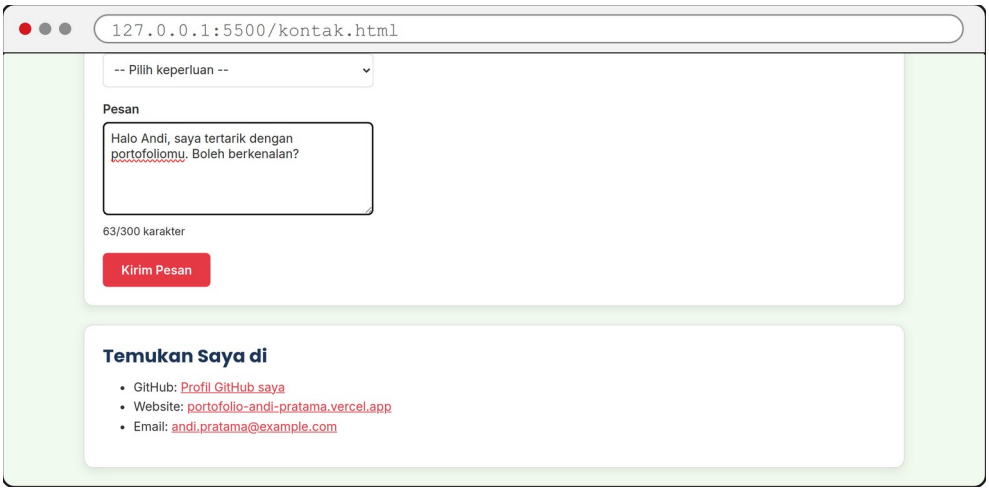
Tambahkan bagian 3 di akhir `main.js` pada Kode 5.13. `kotakPesan.value.length` adalah jumlah karakter yang sedang diketik, karena teks juga punya `length` seperti array.

Kode 5.13 Bagian 3 `main.js`: penghitung karakter

```
// 3. Penghitung karakter pesan (halaman Kontak)
const kotakPesan = document.querySelector("#pesan");
const hitungKarakter = document.querySelector("#hitung-karakter");

if (kotakPesan) {
  kotakPesan.addEventListener("input", () => {
    hitungKarakter.textContent = `${kotakPesan.value.length}/300
karakter`;
  });
}
```

Buka halaman Kontak dan ketik pesan. Angka penghitung berubah di setiap huruf seperti pada Gambar 5.10.



Gambar 5.10 Penghitung karakter pesan di halaman Kontak

5.5 Rangkuman

- Event adalah kejadian di halaman; `elemen.addEventListener("jenis", () => { ... })` menjalankan fungsi setiap kali event terjadi.
- `click` terjadi saat elemen diklik; `input` terjadi setiap isi kotak isian berubah, dan isinya dibaca dari `value`.
- Variabel CSS dibuat di `:root` dengan nama `--nama` dan dipakai dengan `var(--nama)`.
- Mode gelap: `classList.toggle("gelap")` pada `body`, lalu `body.gelap` mengisi ulang variabel warna sehingga semua bagian ikut berubah.
- `classList.contains` dipakai untuk memilih teks tombol yang sesuai.

5.6 Latihan

1. Sebutkan tiga bagian `addEventListener` dan fungsi masing-masing.
2. Apa perbedaan event `click` dan `input`? Berikan satu contoh pemakaian masing-masing di website.
3. Mengapa mode gelap lebih mudah dibuat dengan variabel CSS daripada menulis ulang semua warna?
4. Tuliskan aturan CSS yang membuat variabel `--warna-tombol` berisi `#E63946` di `:root`, lalu memakainya sebagai warna latar `button`.
5. Apa hasil `document.body.classList.contains("gelap")` setelah tombol mode gelap diklik dua kali? Jelaskan.

BAB VI
KEGIATAN PEMBELAJARAN 5: DAFTAR PROYEK DINAMIS DAN FILTER

6.1 Tujuan Kegiatan Pembelajaran

Setelah kegiatan ini kamu mampu:

1. menyusun data proyek sebagai array of objects dengan properti kategori;
2. membuat kartu proyek dari data dengan *template literal* beberapa baris dan `innerHTML`;
3. memakai `map` dan `join` untuk mengubah banyak data menjadi banyak kartu;
4. menyaring proyek per kategori dengan `filter` dan tombol;
5. menyimpan perubahan minggu ini ke GitHub dengan *commit* dan *push*.

6.2 Uraian Materi

6.2.1 Data Proyek dalam Satu Array

Halaman Proyek saat ini berisi tiga kartu yang ditulis langsung di HTML. Setiap kali ada proyek baru, kamu harus menyalin satu blok `article` lengkap dan berisiko salah tulis. Jika semua proyek disimpan sebagai array of objects, menambah proyek cukup dengan menambah satu object, dan HTML kartunya dibuat JavaScript dengan pola yang sama.

Minggu ini halaman Proyek berisi enam proyek dari tiga kategori, seperti pada Tabel 6.1. Tiga proyek baru berasal dari pekerjaanmu sendiri: portofolio responsif dari Minggu 03 dan 04, serta kalkulator nilai dan lima soal logika dari Minggu 06.

Tabel 6.1 Enam proyek di halaman Proyek

No	Judul	Kategori	Gambar
1	Halaman Profil Pertama	HTML	`proyek-profil.png`
2	Form Kontak	HTML	`proyek-kontak.png`
3	Tabel Jadwal Belajar	HTML	`proyek-jadwal.png`
4	Portofolio Responsif	CSS	`proyek-responsif.png` (baru)
5	Kalkulator Nilai Rapor	JavaScript	`proyek-kalkulator.png` (baru)
6	Lima Soal Logika	JavaScript	`proyek-logika.png` (baru)

6.2.2 Membuat Kartu dari Data

Template literal dari Minggu 06 boleh ditulis dalam **beberapa baris**. Kemampuan ini sangat cocok untuk menulis HTML kartu, karena isi dari object disisipkan dengan `${...}`. Fungsi pada Kode 6.1 menerima satu object proyek dan mengembalikan teks HTML satu kartu.

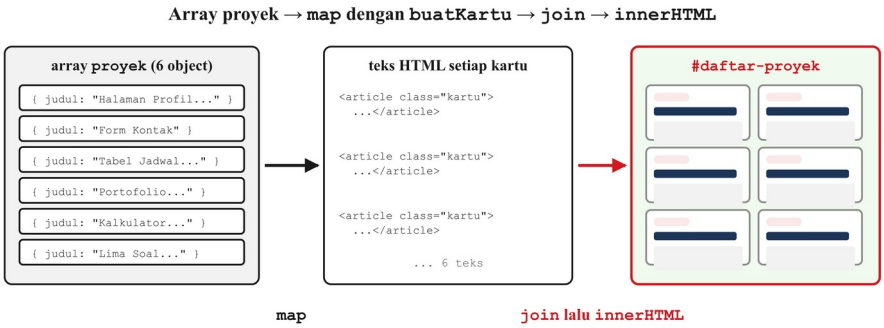
Kode 6.1 Fungsi `buatKartu`

```
function buatKartu(item) {
  return `
    <article class="kartu">
      <span class="label">${item.kategori}</span>
      <h3>${item.judul}</h3>
      <img src="${item.gambar}" alt="${item.alt}" width="280">
      <p>${item.deskripsi}</p>
    </article>`;
}
```

Teks HTML itu kemudian dimasukkan ke halaman dengan `innerHTML`. Karena datanya milikmu sendiri, `innerHTML` aman dipakai di sini.

6.2.3 `map` dan `join`: Banyak Data Menjadi Banyak Kartu

Untuk enam proyek, `buatKartu` harus dijalankan enam kali. `map` dari Kegiatan Pembelajaran 2 melakukannya sekaligus dan menghasilkan array berisi enam teks HTML. Method `join("")` lalu menyambung semua teks itu menjadi satu teks panjang tanpa pemisah, siap dimasukkan ke `innerHTML`. Alur lengkapnya terlihat pada Gambar 6.1.

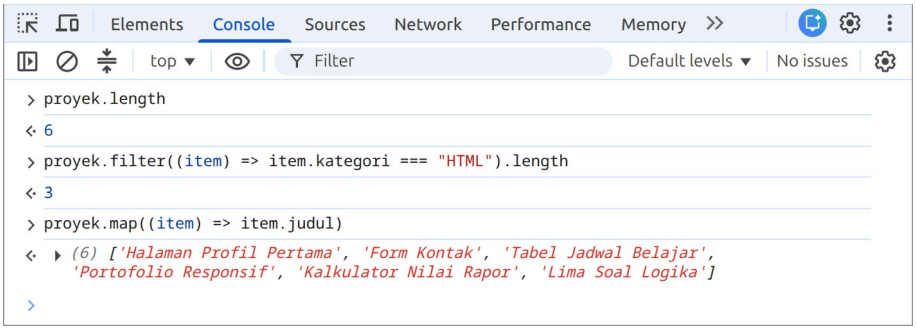


Saat filter dipilih, array yang sudah disaring melewati jalur yang sama, sehingga kartunya ikut berganti.

Gambar 6.1 Alur dari data proyek ke kartu di halaman

6.2.4 `filter` untuk Memilih Kategori

Menyaring proyek per kategori cukup dengan `filter`: pilih object yang `kategori`-nya sama dengan kategori tombol. Array hasil saringan lalu dikirim ke jalur yang sama dengan Gambar 6.1, sehingga kartunya ikut berganti. Setelah praktik selesai, kamu bisa mencoba data proyek langsung di Console seperti pada Gambar 6.2.



Gambar 6.2 Mencoba data proyek dari Console

6.3 Praktik: Halaman Proyek Dinamis

Praktik ini mengubah halaman Proyek dari tiga kartu yang ditulis manual menjadi enam kartu yang dibuat dari data. Urutannya: menyiapkan gambar, mengosongkan HTML, menulis data, menampilkan kartu, lalu memasang tombol filter. Periksa halaman Proyek dan Console setiap selesai satu tahap.

6.3.1 Tahap 1: Menyiapkan Gambar Proyek Baru

Siapkan tiga tangkapan layar milikmu sendiri dan simpan di folder `img` dengan nama pada Tabel 6.2. Ukuran yang disarankan sekitar 640 x 420 piksel, sama dengan gambar proyek lama; potong gambar dengan aplikasi Paint atau Photos jika perlu.

Tabel 6.2 Tiga gambar proyek baru

No	Nama File	Isi Tangkapan Layar
1	`proyek-responsif.png`	Portofoliomu di mode perangkat, misalnya Beranda di lebar 360 dan 1280 piksel
2	`proyek-kalkulator.png`	Console hasil `latihan.js` Minggu 06 (rata-rata, predikat, dan rapor)
3	`proyek-logika.png`	Console hasil uji lima soal `logika.js` Minggu 06

> **Tips**
> File `latihan.js` dan `logika.js` sudah dilepas dari `index.html` di Kegiatan Pembelajaran 1. Untuk memotret hasilnya, pasang sementara kembali baris `script`-nya, ambil tangkapan layar, lalu hapus lagi.

6.3.2 Tahap 2: Wadah Kartu dan Tombol Filter

Di `proyek.html`, ganti `div .daftar-kartu` beserta ketiga `article` di dalamnya dengan kode pada Kode 6.2. Kartu tidak lagi ditulis di HTML; yang tersisa hanya dua tombol filter, paragraf jumlah, dan wadah kartu yang kosong.

Kode 6.2 Tombol filter dan wadah kartu di `proyek.html`

```
<div class="filter">
  <button type="button" class="tombol-filter" id="filter-
semua">Semua</button>
  <button type="button" class="tombol-filter" id="filter-
javascript">JavaScript</button>
</div>
<p id="jumlah-proyek"></p>

<div class="daftar-kartu" id="daftar-proyek"></div>
```

Perbarui juga jawaban FAQ pertama di halaman Proyek, karena proyekmu kini tidak hanya HTML, menjadi seperti pada Kode 6.3.

Kode 6.3 Jawaban FAQ yang diperbarui

```
<p>Proyek di halaman ini dibuat dengan HTML, CSS, dan JavaScript.
Pilih tombol kategori di atas untuk melihat proyek per bahasa.</p>
```

Lalu tambahkan gaya tombol filter dan label kategori ke bagian 10 `style.css` pada Kode 6.4.

Kode 6.4 Gaya tombol filter dan label kategori di bagian 10

```
.filter {
  display: flex;
  flex-wrap: wrap;
  gap: 0.5rem;
  margin-bottom: 1rem;
}

.tombol-filter {
  padding: 0.5rem 1rem;
  border: 2px solid var(--warna-garis);
  background-color: var(--warna-kartu);
  color: var(--warna-judul);
}

.label {
  display: inline-block;
  padding: 0.1rem 0.6rem;
  margin-bottom: 0.5rem;
  border-radius: 999px;
  background-color: var(--warna-latar);
  color: var(--warna-judul);
  font-size: 0.85rem;
  font-weight: 600;
}
```

6.3.3 Tahap 3: Data Proyek

Tambahkan bagian 4 di akhir `main.js` berisi array `proyek`. Karena panjang, kodenya dibagi dua. Tiga proyek pertama terlihat pada Kode 6.5.

Kode 6.5 Bagian 4 `main.js`: data proyek (1 sampai 3)

```
// 4. Daftar proyek dan filter (halaman Proyek)
const proyek = [
  {
    judul: "Halaman Profil Pertama",
    kategori: "HTML",
    gambar: "img/proyek-profil.png",
    alt: "Tangkapan layar halaman profil Andi Pratama",
    deskripsi: "Halaman web pertama saya, dibuat dengan HTML
dasar.",
  },
  {
    judul: "Form Kontak",
    kategori: "HTML",
    gambar: "img/proyek-kontak.png",
    alt: "Tangkapan layar form kontak berisi lima isian",
    deskripsi: "Form dengan label dan validasi bawaan browser.",
  },
  {
    judul: "Tabel Jadwal Belajar",
    kategori: "HTML",
    gambar: "img/proyek-jadwal.png",
    alt: "Tangkapan layar tabel jadwal belajar mandiri",
    deskripsi: "Jadwal belajar mingguan dengan thead dan tbody.",
  },
```

Tiga proyek berikutnya beserta tutup array terlihat pada Kode 6.6. Ketik tepat di bawah kode sebelumnya.

Kode 6.6 Data proyek (4 sampai 6)

```
  {
    judul: "Portofolio Responsif",
    kategori: "CSS",
    gambar: "img/proyek-responsif.png",
    alt: "Tangkapan layar portofolio di ponsel dan laptop",
    deskripsi: "Rapi di ponsel, tablet, dan laptop dengan Flexbox.",
  },
  {
    judul: "Kalkulator Nilai Rapor",
    kategori: "JavaScript",
    gambar: "img/proyek-kalkulator.png",
    alt: "Tangkapan layar hasil kalkulator nilai di Console",
    deskripsi: "Rata-rata dan predikat nilai dengan fungsi
JavaScript.",
  },
  {
    judul: "Lima Soal Logika",
    kategori: "JavaScript",
    gambar: "img/proyek-logika.png",
    alt: "Tangkapan layar hasil uji lima soal logika",
    deskripsi: "Lima fungsi logika yang masing-masing diuji 3
masukan.",
  },
];
```

6.3.4 Tahap 4: Menampilkan Kartu

Di bawah array `proyek`, ambil wadah kartu dan paragraf jumlah, lalu tulis fungsi `buatKartu` dan `tampilkanProyek` seperti pada Kode 6.7.

Kode 6.7 Fungsi untuk menampilkan kartu proyek

```
const daftarProyek = document.querySelector("#daftar-proyek");
const jumlahProyek = document.querySelector("#jumlah-proyek");

function buatKartu(item) {
  return `
    <article class="kartu">
      <span class="label">${item.kategori}</span>
      <h3>${item.judul}</h3>
      <img src="${item.gambar}" alt="${item.alt}" width="280">
      <p>${item.deskripsi}</p>
    </article>`;
}

function tampilkanProyek(daftar) {
  const semuaKartu = daftar.map((item) => buatKartu(item));
  daftarProyek.innerHTML = semuaKartu.join("");
  jumlahProyek.textContent = `Menampilkan ${daftar.length} dari $
{proyek.length} proyek`;
}
```

- `tampilkanProyek` menerima array apa pun, baik semua proyek maupun hasil saringan, lalu menampilkannya.
- `semuaKartu` adalah array berisi teks HTML setiap kartu hasil `map`, lalu `join("")` menyambungnya.
- Paragraf jumlah membandingkan banyaknya kartu yang tampil (`daftar.length`) dengan seluruh proyek (`proyek.length`).

6.3.5 Tahap 5: Menjalankan Filter

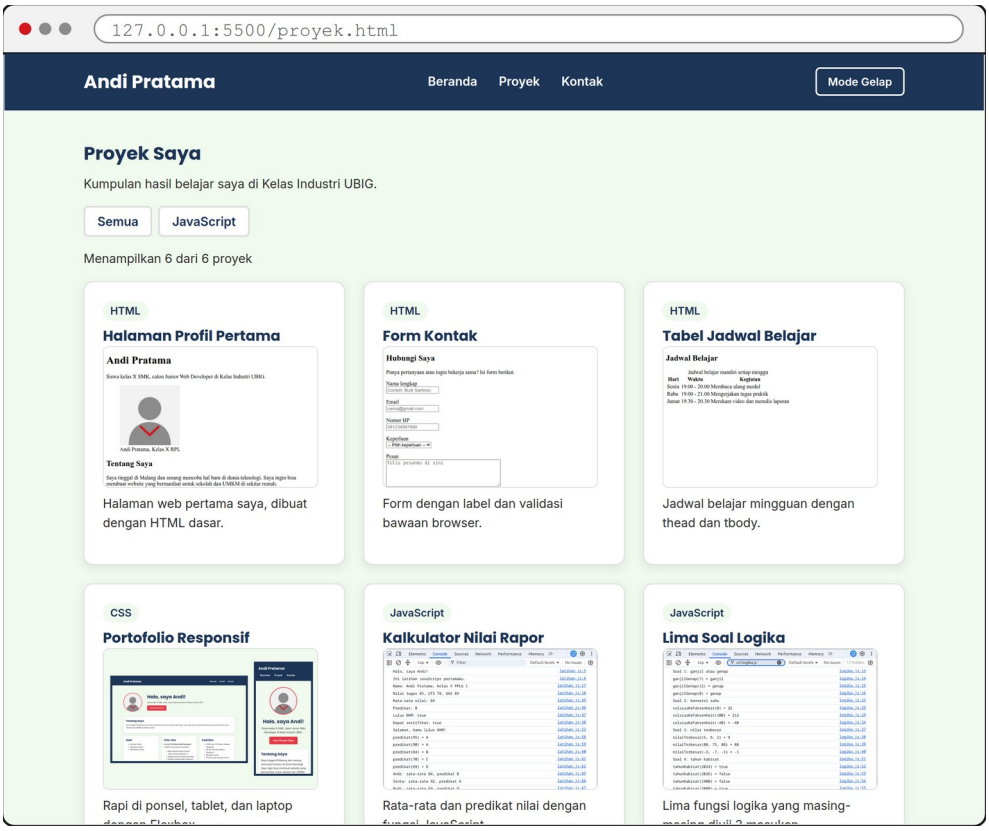
Fungsi `pasangFilter` pada Kode 6.8 memasang event `click` pada satu tombol. Saat diklik, ia menampilkan semua proyek atau hanya proyek dengan kategori tertentu. Di bawahnya, kode di dalam `if (daftarProyek)` menampilkan semua proyek saat halaman dibuka dan memasang dua tombol filter.

Kode 6.8 Fungsi `pasangFilter` dan kode yang menjalankannya

```
function pasangFilter(idTombol, kategori) {
  const tombol = document.querySelector(idTombol);
  tombol.addEventListener("click", () => {
    if (kategori === "Semua") {
      tampilkanProyek(proyek);
    } else {
      tampilkanProyek(proyek.filter((item) => item.kategori ===
kategori));
    }
  });
}

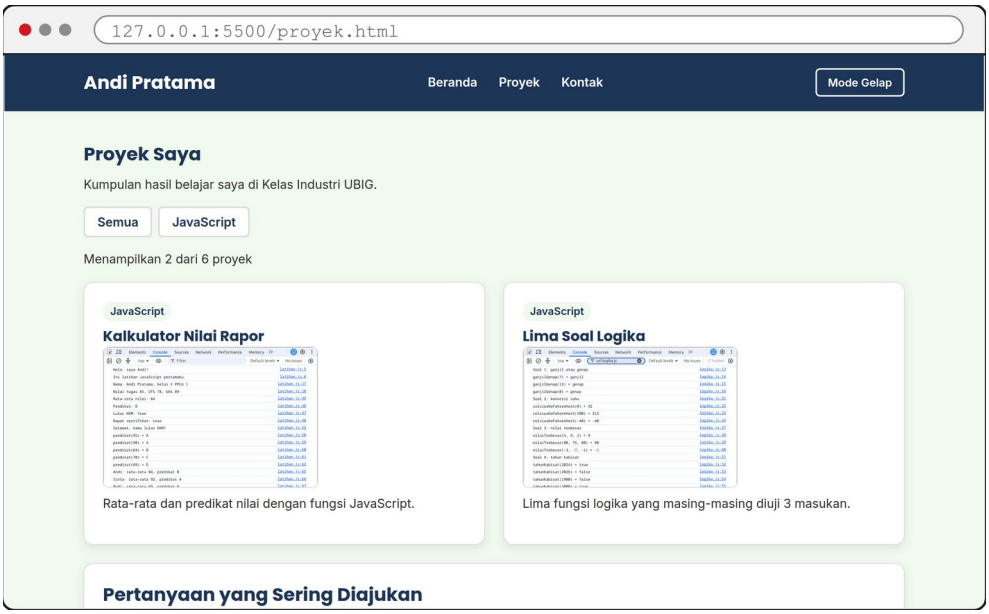
if (daftarProyek) {
  tampilkanProyek(proyek);
  pasangFilter("#filter-semua", "Semua");
  pasangFilter("#filter-javascript", "JavaScript");
}
```

Simpan, lalu buka halaman Proyek. Keenam kartu tampil beserta label kategorinya dan tulisan `Menampilkan 6 dari 6 proyek`, seperti pada Gambar 6.3.



Gambar 6.3 Halaman Proyek berisi enam kartu dari array

Klik tombol **JavaScript**. Hanya dua kartu yang tersisa dan tulisannya menjadi `Menampilkan 2 dari 6 proyek`, seperti pada Gambar 6.4. Klik **Semua** untuk menampilkan keenam kartu lagi.



Gambar 6.4 Filter JavaScript menampilkan dua proyek

Di layar ponsel, kartu tersusun satu kolom dan tombol filter tetap rapi berkat `flex-wrap`, seperti pada Gambar 6.5.



Gambar 6.5 Halaman Proyek dinamis di layar ponsel

6.3.6 Tahap 6: Kode Lengkap `main.js`

Isi `main.js` sekarang terdiri atas empat bagian: keahlian, tema, penghitung karakter, dan proyek. Bagian 1 sampai 3 terlihat pada Kode 6.9. Bagian 4 adalah gabungan Kode 6.5, Kode 6.6, Kode 6.7, dan Kode 6.8 secara berurutan.

Kode 6.9 Isi `js/main.js` bagian 1 sampai 3

```
// JavaScript portofolio Andi Pratama
// Dipasang di ketiga halaman dengan defer

// 1. Daftar keahlian (halaman Beranda)
const skills = [
  { nama: "HTML", level: "Menengah" },
  { nama: "CSS", level: "Menengah" },
  { nama: "Git dan GitHub", level: "Dasar" },
  { nama: "Desain poster dengan Canva", level: "Mahir" },
```

```
];
skills.push({ nama: "JavaScript", level: "Dasar" });

const daftarSkill = document.querySelector("#daftar-skill");
const jumlahSkill = document.querySelector("#jumlah-skill");

if (daftarSkill) {
  skills.forEach((skill) => {
    const item = document.createElement("li");
    item.textContent = `${skill.nama} (${skill.level})`;
    daftarSkill.appendChild(item);
  });
  jumlahSkill.textContent = `Ada ${skills.length} keahlian yang
sedang saya kembangkan.`;
}

// 2. Tema gelap (semua halaman)
const tombolTema = document.querySelector("#tombol-tema");

tombolTema.addEventListener("click", () => {
  document.body.classList.toggle("gelap");

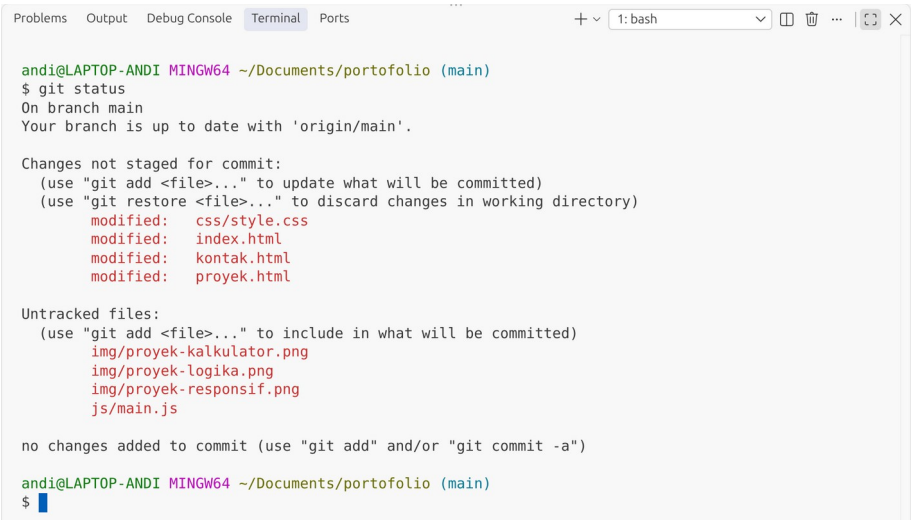
  if (document.body.classList.contains("gelap")) {
    tombolTema.textContent = "Mode Terang";
  } else {
    tombolTema.textContent = "Mode Gelap";
  }
});

// 3. Penghitung karakter pesan (halaman Kontak)
const kotakPesan = document.querySelector("#pesan");
const hitungKarakter = document.querySelector("#hitung-karakter");

if (kotakPesan) {
  kotakPesan.addEventListener("input", () => {
    hitungKarakter.textContent = `${kotakPesan.value.length}/300
karakter`;
  });
}
```

6.3.7 Tahap 7: Commit dan Push

Buka terminal VS Code dan jalankan `git status`. Git mencatat tiga halaman HTML dan `style.css` yang berubah, serta `js/main.js` dan tiga gambar baru yang belum dilacak, seperti pada Gambar 6.6.



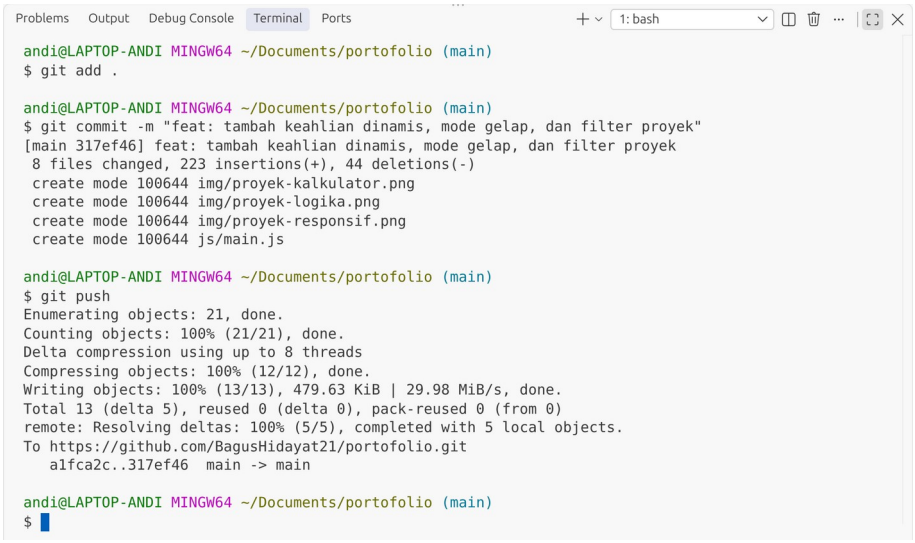
Gambar 6.6 `git status` setelah praktik Minggu 07

Simpan semuanya dalam satu *commit*, lalu kirim ke GitHub dengan perintah pada Kode 6.10.

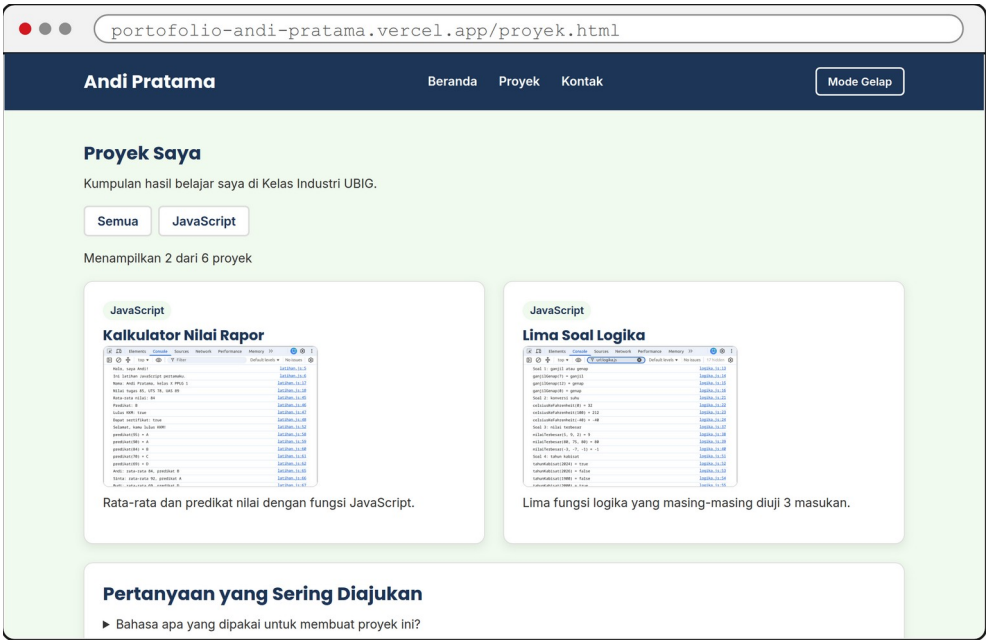
Kode 6.10 Menyimpan dan mengirim praktik Minggu 07

```
$ git add .
$ git commit -m "feat: tambah keahlian dinamis, mode gelap, dan
filter proyek"
$ git push
```

Hasil *push* terlihat pada Gambar 6.7. Setelah Vercel selesai memperbarui website, buka halaman Proyek di alamat `vercel.app` milikmu dan coba filternya seperti pada Gambar 6.8.



Gambar 6.7 *Commit* dan *push* praktik Minggu 07



Gambar 6.8 Filter proyek di website *live*

> **Kesalahan Umum**
>
> Website *live* sudah memuat HTML terbaru, tetapi tombol atau filter masih bekerja seperti versi lama? *Browser* menyimpan salinan file JavaScript dan CSS lama di *cache* agar halaman cepat dibuka. Tekan **Ctrl+F5** (atau **Ctrl+Shift+R**) untuk memuat ulang tanpa *cache*. Saat DevTools terbuka, kamu juga bisa mencentang **Disable cache** di tab Network.

6.4 Memeriksa Hasil Kerja

Sebelum mengerjakan tugas, periksa hasil praktikmu dengan daftar pada Tabel 6.3.

Tabel 6.3 Cara memeriksa hasil kerja Minggu 07

No	Yang Diperiksa	Cara Memeriksa
1	Keahlian dari array	Kartu Keahlian menampilkan lima keahlian; di `index.html` dan **Ctrl+U**, `ul#daftar-skill` tetap kosong
2	Mode gelap di semua bagian	Klik tombol di ketiga halaman; latar, kartu, judul, teks, tabel, form, dan garis berubah, teks tetap terbaca
3	Penghitung karakter	Ketik di kotak pesan Kontak; angka berubah di setiap huruf dan berhenti di 300
4	Proyek dari array	Halaman Proyek menampilkan enam kartu berlabel; tombol JavaScript menampilkan 2 dari 6 proyek
5	Console bersih	Tidak ada *error* merah di ketiga halaman, termasuk `Cannot read properties of null`

6.5 Rangkuman

- Data proyek disimpan sebagai array of objects dengan properti `judul`, `kategori`, `gambar`, `alt`, dan `deskripsi`.
- *Template literal* boleh ditulis beberapa baris dan cocok untuk membuat HTML kartu dari data.
- `map` mengubah setiap object menjadi teks HTML, `join("")` menyambungnya, lalu `innerHTML` menampilkannya.
- `filter` memilih proyek per kategori; fungsi yang sama menampilkan hasilnya.
- Satu fungsi `pasangFilter` dipakai untuk banyak tombol dengan parameter yang berbeda.

6.6 Latihan

1. Apa keuntungan menyimpan data proyek dalam array dibandingkan menulis setiap kartu di HTML?
2. Apa hasil `["<li>A</li>", "<li>B</li>"].join("")`?
3. Tuliskan kode `filter` untuk mengambil proyek berkategori `"CSS"` dari array `proyek`, lalu sebutkan jumlah hasilnya.

4. Mengapa `tampilkanProyek` dibuat menerima parameter `daftar`, bukan langsung memakai `proyek`?
5. Tuliskan satu baris pemanggilan `pasangFilter` untuk tombol dengan `id="filter-html"` dan kategori `"HTML"`.

6.7 Tugas dan Pengumpulan Minggu Ini

Bagian ini hanya memuat tugas dan ketentuan pengumpulan yang khusus untuk Minggu 07. Format laporan lengkap, standar video, tenggat, dan sanksi yang berlaku setiap minggu tercantum di dokumen **Panduan dan Ketentuan Kelas Industri UBIG**.

6.7.1 Tugas Rumah

Selesaikan filter kartu proyek berdasarkan kategori dengan ketentuan berikut:

1. tambahkan tombol **HTML** dan **CSS** di antara tombol **Semua** dan **JavaScript**, dengan `id` `filter-html` dan `filter-css`;
2. pasang kedua tombol baru dengan `pasangFilter`, sehingga setiap kategori menampilkan jumlah kartu yang benar sesuai Tabel 6.4;
3. tandai tombol yang sedang dipilih dengan class `aktif`: tombol **Semua** diberi class `aktif` di HTML, lalu setiap kali tombol diklik, class `aktif` dibuang dari tombol sebelumnya dan ditambahkan ke tombol yang diklik;
4. beri gaya `.tombol-filter.aktif` di bagian 10 `style.css` agar tombol yang dipilih tampak berbeda di tema terang dan gelap;
5. lakukan *commit* dan *push*, ambil 3 tangkapan layar sesuai Tabel 6.7, rekam video, lalu kirim laporan.

Tabel 6.4 Hasil filter yang benar

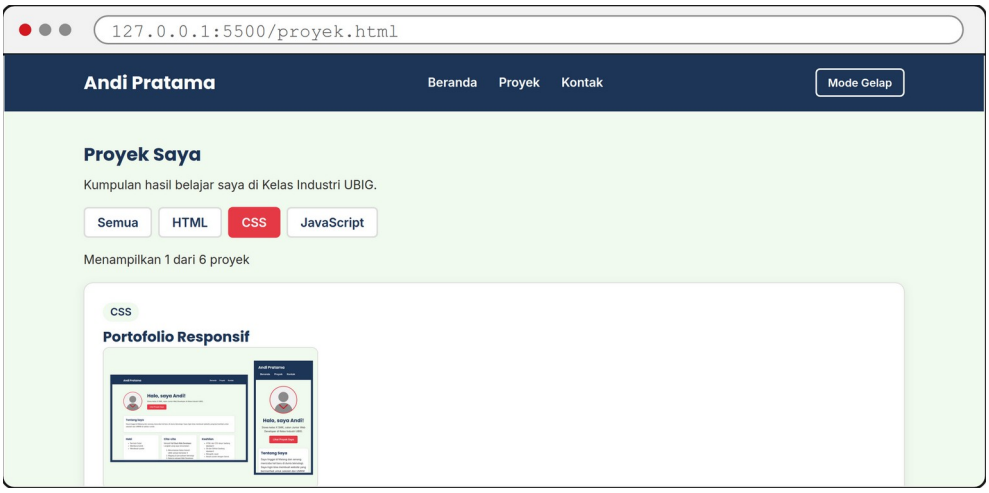
No	Tombol	Kartu yang Tampil	Tulisan Jumlah
1	Semua	Keenam proyek	`Menampilkan 6 dari 6 proyek`
2	HTML	Profil, Form Kontak, Tabel Jadwal	`Menampilkan 3 dari 6 proyek`
3	CSS	Portofolio Responsif	`Menampilkan 1 dari 6 proyek`
4	JavaScript	Kalkulator Nilai, Lima Soal Logika	`Menampilkan 2 dari 6 proyek`

Untuk menandai tombol aktif, kamu membutuhkan dua hal yang sudah dipelajari: `querySelector` dengan *selector* gabungan dan `classList`. Petunjuknya ada pada Tabel 6.5.

Tabel 6.5 Petunjuk menandai tombol aktif

No	Kode	Kegunaan
1	`document.querySelector(".tombol-filter.aktif")`	Mengambil tombol filter yang saat ini memiliki class `aktif` (dua class tanpa spasi)
2	`.classList.remove("aktif")`	Membuang class `aktif` dari tombol itu
3	`tombol.classList.add("aktif")`	Menambahkan class `aktif` ke tombol yang baru diklik; tulis di dalam fungsi `click` pada `pasangFilter`

Hasil yang diharapkan saat tombol **CSS** diklik terlihat pada Gambar 6.9: hanya tombol CSS yang berwarna, dan hanya satu kartu yang tampil.



Gambar 6.9 Contoh hasil tugas: filter CSS dengan tombol aktif

6.7.2 Tantangan (Opsional)

Tantangan: ingat pilihan tema. Saat ini tema kembali terang setiap kali pindah halaman atau memuat ulang. Simpan pilihan tema di **`localStorage`**, yaitu penyimpanan kecil milik *browser* yang tetap ada walaupun halaman ditutup. Petunjuknya ada pada Tabel 6.6. Tantangan berhasil jika tema gelap tetap berlaku saat berpindah dari Beranda ke Proyek dan setelah halaman dimuat ulang.

Tabel 6.6 Petunjuk tantangan `localStorage`

No	Kode	Kegunaan
1	`localStorage.setItem("tema", "gelap")`	Menyimpan teks `"gelap"` dengan nama `"tema"`; jalankan setiap tombol diklik sesuai tema yang baru
2	`localStorage.getItem("tema")`	Membaca isi `"tema"`; hasilnya `"gelap"`, `"terang"`, atau `null` jika belum pernah disimpan
3	`document.body.classList.add("gelap")`	Menerapkan tema gelap saat halaman dibuka jika isi `"tema"` adalah `"gelap"`

Jika kamu mengerjakan tantangan, tuliskan hasilnya sebagai salah satu butir Pencapaian Praktikum di laporan.

6.7.3 Kriteria Selesai

Tugasmu dianggap selesai jika memenuhi semua kriteria berikut:

• daftar skill berasal dari array, bukan ditulis di HTML;

• *dark mode* berlaku di semua bagian halaman;

• filter menampilkan jumlah kartu sesuai kategori.

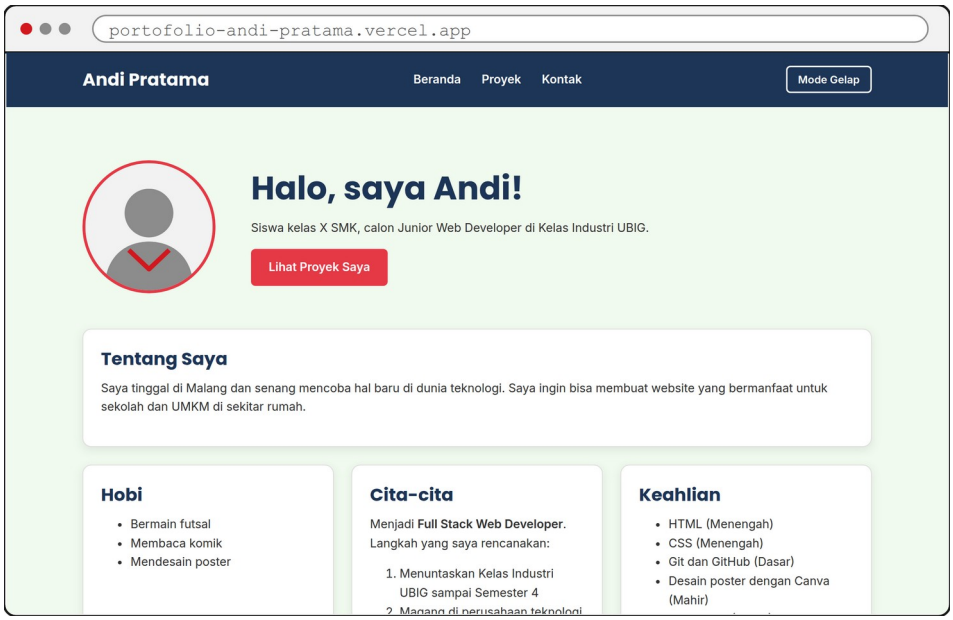
6.7.4 Ketentuan Pengumpulan P1-W07

Isi laporan, tangkapan layar, dan video minggu ini mengikuti Tabel 6.7.

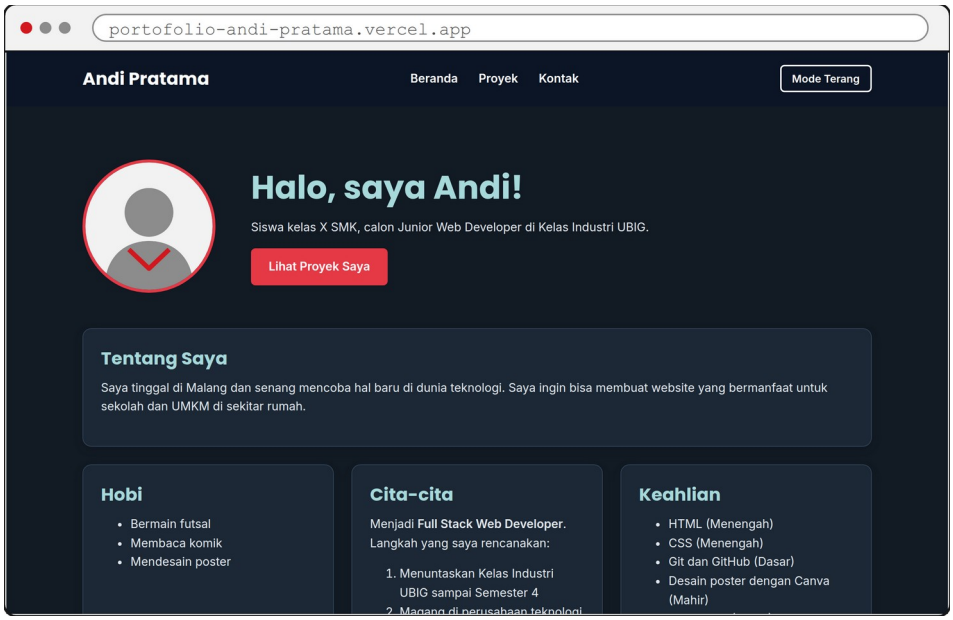
Tabel 6.7 Ketentuan pengumpulan P1-W07

Aspek	Ketentuan Minggu Ini
Kode minggu	`P1-W07`
Baris *Project* di laporan	`Project: Portofolio Pribadi – Modul P1-W07: Array, Object, DOM, dan Event`
Tangkapan layar	1. Beranda mode terang dengan kartu Keahlian terlihat: `P1W07_NamaSiswa_01_terang.png` 2. Beranda yang sama dalam mode gelap: `P1W07_NamaSiswa_02_gelap.png` 3. Halaman Proyek di alamat `vercel.app` dengan filter JavaScript aktif: `P1W07_NamaSiswa_03_filter.png`
Link GitHub dan Vercel	Tautan repositori `portofolio` dan website `vercel.app` (wajib sejak Minggu 05)
Judul video	`[UBIG P1-W07] DOM Manipulation and Events | [Nama Lengkap] – [Nama Sekolah]`
Isi demo video	Tunjukkan daftar keahlian lalu tambahkan satu keahlian di array, klik tombol mode gelap di ketiga halaman, ketik di kotak pesan Kontak, klik keempat tombol filter, lalu jelaskan `forEach`, `addEventListener`, dan `filter` di `main.js`
Tenggat	H-1 sebelum pertemuan Minggu 08, pukul 22.00 WIB

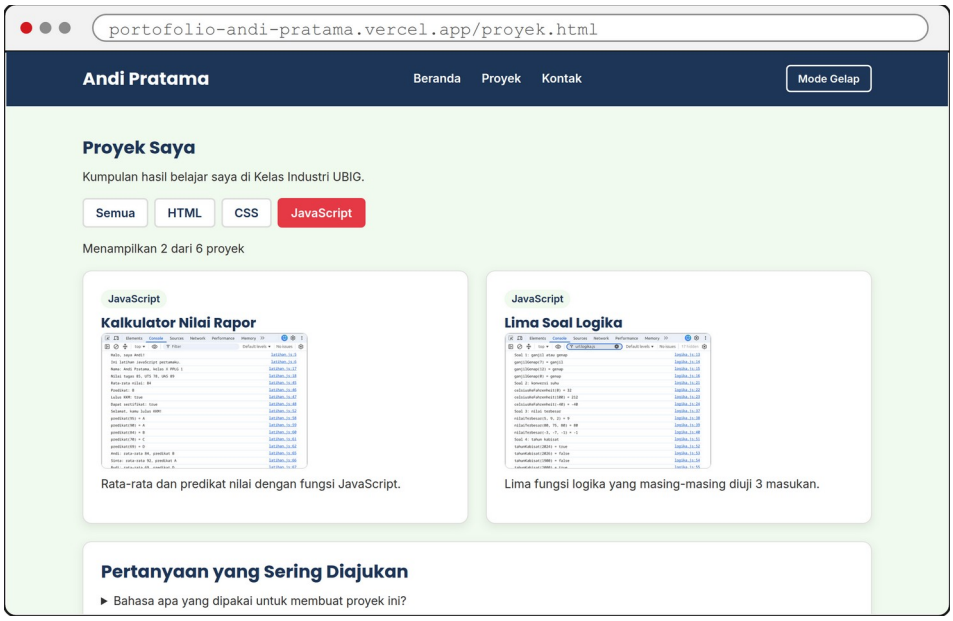
Contoh tangkapan layar pertama dan kedua terlihat pada Gambar 6.10 dan Gambar 6.11, sedangkan contoh ketiga pada Gambar 6.12.



Gambar 6.10 Contoh tangkapan layar pertama: mode terang



Gambar 6.11 Contoh tangkapan layar kedua: mode gelap



Gambar 6.12 Contoh tangkapan layar ketiga: filter di website *live*

6.7.5 Urutan Mengerjakan Tugas

1. Selesaikan praktik Kegiatan Pembelajaran 1 sampai 5, lalu periksa dengan Tabel 6.3.
2. Lengkapi tombol filter dan tanda tombol aktif, lalu cocokkan hasilnya dengan Tabel 6.4.
3. Lakukan *commit* dan *push*, misalnya dengan pesan `feat: lengkapi filter kategori proyek`, lalu periksa website *live*.

4. Ambil 3 tangkapan layar sesuai Tabel 6.7.
5. Rekam video, unggah ke YouTube sebagai *Unlisted*, dan masukkan ke *playlist* Paket 1.
6. Tulis laporan dengan format baku di Panduan, isi Link GitHub, Link Vercel, dan Link YouTube, lalu kirim bersama 3 tangkapan layar melalui Telegram sebelum tenggat.
7. Pelajari ulang modul dan soal pilihan ganda serta uraian di bab Evaluasi untuk kuis di awal pertemuan P1-W08.

BAB VII
EVALUASI

Soal pilihan ganda dan uraian berikut menjadi **kuis 10 menit** di awal pertemuan P1-W08, dikerjakan perorangan tanpa membuka modul melalui Google Form dari mentor. Pelajari ulang seluruh Kegiatan Pembelajaran sebelum pertemuan itu.

Soal praktik wajib dikerjakan sebagai **remedial** jika tugas minggu ini belum memenuhi kriteria selesai. Jika sudah memenuhi, soal praktik menjadi latihan menjelang uji kompetensi Minggu 16. Ketentuan lengkap kuis dan remedial tercantum di dokumen **Panduan dan Ketentuan Kelas Industri UBIG** bab Penilaian.

7.1 Soal Pilihan Ganda

Pilih satu jawaban yang paling tepat.

1. Diketahui `const hari = ["Senin", "Selasa", "Rabu"];`. Nilai `hari[1]` adalah ....
   a. `"Senin"`
   b. `"Selasa"`
   c. `"Rabu"`
   d. `undefined`
2. Method untuk menambahkan isi baru di akhir array adalah ....
   a. `length`
   b. `filter`
   c. `push`
   d. `join`
3. Diketahui `const siswa = { nama: "Sinta", kelas: "X PPLG 1" };`. Cara membaca kelas Sinta adalah ....
   a. `siswa[kelas]`
   b. `siswa.kelas`
   c. `kelas.siswa`
   d. `siswa("kelas")`
4. Hasil `[10, 20, 30].map((n) => n * 2)` adalah ....

a. `[20, 40, 60]`

b. `[10, 20, 30]`

c. `60`

d. `[20]`

5. Hasil `[55, 80, 92].filter((n) => n >= 75)` adalah ....

   a. `[55]`

   b. `[80]`

   c. `true`

   d. `[80, 92]`

6. Jika tidak ada elemen yang cocok, `document.querySelector("#tidak-ada")` menghasilkan ....

   a. `undefined`

   b. `false`

   c. `null`

   d. `0`

7. Properti yang mengganti isi elemen dengan teks biasa (tag tidak dibaca sebagai HTML) adalah ....

   a. `textContent`

   b. `innerHTML`

   c. `value`

   d. `classList`

8. Urutan yang benar untuk menambahkan `li` baru ke halaman adalah ....

   a. `appendChild`, lalu `createElement`, lalu isi teks

   b. `createElement`, isi teks, lalu `appendChild`

   c. isi teks, `appendChild`, lalu `createElement`

   d. cukup `createElement` saja

9. Event yang terjadi setiap kali isi kotak isian berubah saat diketik adalah ....

   a. `click`

   b. `submit`

   c. `load`

   d. `input`

10. `document.body.classList.toggle("gelap")` berfungsi untuk ....
    a. menambahkan class `gelap` jika belum ada dan membuangnya jika sudah ada
    b. selalu menambahkan class `gelap`
    c. selalu membuang class `gelap`
    d. memeriksa apakah class `gelap` ada

7.2 Soal Uraian

1. Jelaskan perbedaan array dan object. Kemudian tuliskan array of objects berisi tiga mata pelajaran dengan properti `nama` dan `jam`, lalu tuliskan kode untuk menampilkan nama mata pelajaran kedua.
2. Halaman memiliki `<ul id="daftar-hobi"></ul>` dan JavaScript memiliki `const hobi = ["Futsal", "Membaca", "Menggambar"];`. Tuliskan kode lengkap untuk menampilkan setiap hobi sebagai `li` di dalam `ul` tersebut, lalu jelaskan fungsi setiap baris.
3. Jelaskan cara kerja mode gelap di portofolio mulai dari tombol diklik sampai warna kartu berubah. Sebutkan peran `addEventListener`, `classList.toggle`, `body.gelap`, dan variabel CSS.

7.3 Soal Praktik

Kembangkan kasir kantin Minggu 06 menjadi halaman menu kantin dinamis di folder baru `menu-kantin` dengan ketentuan:

1. buat `index.html` berjudul `Menu Kantin SMK Mitra` dengan ikon tab, file `css/menu.css`, dan `js/menu.js` yang dipasang dengan `defer`;
2. simpan minimal enam menu dalam array of objects dengan properti `nama`, `harga`, dan `kategori` (`"Makanan"` atau `"Minuman"`), masing-masing minimal tiga;
3. tampilkan setiap menu sebagai `li` berisi nama dan harga, misalnya `Nasi goreng - Rp12000`, dengan `createElement` dan `appendChild`;
4. tampilkan jumlah menu yang sedang tampil, misalnya `6 menu tersedia`;
5. buat tombol **Semua**, **Makanan**, dan **Minuman** yang menyaring menu dengan `filter`;
6. buat tombol mode gelap dengan variabel CSS dan `classList.toggle`;

7. pastikan tidak ada *error* di Console.

7.4 Rubrik Penilaian Tugas

Rubrik ini menilai tugas rumah minggu ini (komponen praktik dan kode), bukan soal praktik di atas. Kriterianya tercantum pada Tabel 7.1. Tantangan bersifat opsional; tanpa tantangan, nilai tertinggi 90.

Tabel 7.1 Rubrik penilaian tugas P1-W07

No	Aspek	Kriteria Nilai Penuh	Skor
1	Keahlian dari array	Daftar keahlian dibuat dari array of objects dengan `createElement`; `ul` di HTML kosong dan jumlah keahlian tampil	15
2	Mode gelap	Tombol di ketiga halaman; variabel CSS membuat semua bagian halaman berubah dan teks tombol menyesuaikan	20
3	Filter proyek	Enam kartu dari array; keempat tombol menampilkan jumlah kartu yang benar sesuai kategori	30
4	Tombol aktif dan jumlah	Class `aktif` berpindah ke tombol yang diklik dan tulisan jumlah sesuai	15
5	Kerapian dan GitHub	Console tanpa *error* di ketiga halaman, kode rapi dengan komentar bagian, dan *commit* sudah di-*push*	10
6	Tantangan	Pilihan tema tersimpan di `localStorage` dan tetap berlaku saat pindah halaman (nilai tambah)	10

BAB VIII
PENUTUP

Selamat, portofoliomu kini benar-benar hidup! Minggu ini kamu menyimpan banyak data dengan array dan object, mengolahnya dengan `forEach`, `map`, dan `filter`, mengubah halaman melalui DOM, serta menanggapi klik dan ketikan dengan event. Hasilnya terlihat langsung: daftar keahlian dan kartu proyek dibuat dari data, mode gelap berlaku di semua bagian halaman, dan form Kontak menghitung karakter pesan.

Data proyekmu masih ditulis di dalam `main.js`. Pada Minggu 08 kamu akan memindahkan data itu ke file JSON, lalu mengambilnya dengan Fetch API, termasuk mengambil daftar repositori GitHub-mu secara otomatis. Pastikan tugas minggu ini selesai, kodenya sudah di-*push*, dan laporan dikirim tepat waktu. Sampai jumpa di pertemuan berikutnya!

GLOSARIUM

Istilah	Pengertian
Array	Kumpulan banyak data berurutan dalam satu variabel, ditulis dengan `[ ]`.
Array of objects	Array yang setiap isinya object dengan properti sejenis.
classList	Properti elemen untuk menambah, membuang, dan memeriksa class.
createElement	Method `document` untuk membuat elemen HTML baru.
DOM	*Document Object Model*, susunan halaman berbentuk pohon di *browser*.
Event	Kejadian di halaman, misalnya klik atau ketikan.
filter	Method array yang membuat array baru berisi isi yang memenuhi syarat.
for...of	Perulangan yang menjalankan blok untuk setiap isi array.
forEach	Method array yang menjalankan fungsi untuk setiap isi.
Indeks	Nomor urut isi array, dimulai dari 0.
innerHTML	Properti untuk mengganti isi elemen dengan teks HTML.
join	Method array yang menyambung semua isi menjadi satu teks.
length	Properti yang menyebutkan banyaknya isi array atau karakter teks.
localStorage	Penyimpanan kecil di *browser* yang tetap ada setelah halaman ditutup.
map	Method array yang membuat array baru dari hasil ubahan setiap isi.
Method	Fungsi milik sebuah object, dipanggil dengan tanda titik.
null	Nilai yang berarti kosong atau tidak ada apa-apa.
Object	Kumpulan pasangan properti dan nilai dalam `{ }`.
Perulangan	Perintah untuk menjalankan blok kode berulang kali.
Properti	Nama keterangan di dalam object, misalnya `nama` atau `level`.
push	Method array untuk menambah isi di akhir array.
querySelector	Method `document` untuk mengambil elemen pertama yang cocok dengan *selector*.
textContent	Properti untuk membaca atau mengganti teks biasa di dalam elemen.
Variabel CSS	Nilai CSS bernama `--nama` yang dipakai dengan `var()`.

DAFTAR PUSTAKA

Haverbeke, M. (2024). *Eloquent JavaScript* (4th ed.). No Starch Press. https://eloquentjavascript.net/

MDN Web Docs. (2026). *Array*. Mozilla. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array

MDN Web Docs. (2026). *Introduction to events*. Mozilla. https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Scripting/Events

MDN Web Docs. (2026). *Manipulating documents*. Mozilla. https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Scripting/DOM_scripting

MDN Web Docs. (2026). *Using CSS custom properties (variables)*. Mozilla. https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_cascading_variables/Using_CSS_custom_properties

MDN Web Docs. (2026). *Window: localStorage property*. Mozilla. https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage

MDN Web Docs. (2026). *Working with objects*. Mozilla. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Working_with_objects